

# swtpcsim — Developer Guide

---

2026-09-27 · 0e4d89e

*A simulation of the SWTPC 6800 on the SS-50/SS-30 bus, and the MITS Altair  
680b*

|                                                      |    |
|------------------------------------------------------|----|
| Building it .....                                    | 1  |
| Theory of operation .....                            | 7  |
| Writing a board .....                                | 21 |
| Serial I/O .....                                     | 32 |
| The built-in terminal .....                          | 39 |
| Soft-sector floppy controllers .....                 | 44 |
| The PS II monitor loader, inside .....               | 51 |
| Adding a built-in ROM ( .S19 → embedded image) ..... | 54 |
| Review comments .....                                | 56 |

# Building it

---

**There are no dependencies.** A C++20 compiler and CMake  $\geq 3.20$  is the entire list. The TOML parser, the JSON encoder and the line editor are all in the tree, so a fresh clone builds with nothing to download.

```
git clone https://github.com/deltecent/swtpcsim.git
cd swtpcsim
cmake -S . -B build && cmake --build build -j
ctest --test-dir build           # the test suite
./build/swtpcsim                 # the default machine
```

If CMake is unfamiliar, the two `cmake` calls are two distinct steps — **configure**, then **build** — and you run both in that order:

- **`cmake -S . -B build`** — *configure*. Reads `CMakeLists.txt` in the source tree (`-S .`, the current directory) and writes the generated build files into a fresh `build/` directory (`-B build`). It downloads nothing and compiles nothing; it just works out *how* to build. You rerun it only when `CMakeLists.txt` itself changes — and even then the build step reruns it for you.
- **`cmake --build build -j`** — *build*. Compiles what the configure step laid out under `build/`. `-j` runs the compiles in parallel across all CPU cores (drop it for a serial build, or `-j4` to cap the jobs). This is the command you repeat after editing code.
- **`ctest --test-dir build`** — run the tests that the build produced. `--test-dir build` points at the same directory.

Everything lands in `build/`, which is disposable: `rm -rf build` and rerun the configure step for a clean slate.

That is not a boast about minimalism for its own sake. It is a property worth defending: the day this needs a package manager to build is the day it stops being something a person can pick up in ten years and compile.

**After a `git pull`, the commands do not change.** `cmake -S . -B build && cmake --build build` again — there is no separate incremental procedure and no need to delete `build/`. New `.cpp` files can't be missed because sources are listed in `CMakeLists.txt` rather than globbed, so adding one edits `CMakeLists.txt` and forces a reconfigure; `roms/` and `machines/` are globbed but use `CONFIGURE_DEPENDS`, so they re-glob on every build. The build directory does cache absolute paths, though, so **rename the checkout or switch compilers and you must `rm -rf build`** — it fails with a loud `CMakeCache.txt` directory is different error, not a silent wrong answer. If a build breaks — or a test that used to pass fails — immediately after a pull, suspect a stale `build/` before suspecting the code: a `rm -rf build` and a clean reconfigure is the first thing to rule out, because a half-rebuilt object can walk an already-fixed bug back in. `docs/building-linux.md` §7 has the details.

## Warnings as errors before a PR

The build always compiles with `-Wall -Wextra -Wpedantic` (GCC/Clang) or `/W4 /permissive-` (MSVC), but by default a warning is just noise and does not fail anything. **CI configures every leg with `-DWERROR=on`**, which on GCC and Clang adds `-Werror` and turns a warning into a build failure — so a PR that introduces one on either of those goes red before it can merge. Reproduce that gate locally before you open the PR:

```
cmake -B build -DWERROR=on && cmake --build build -j
```

It is off by default ( `option(WERROR ... OFF)` in `CMakeLists.txt` ) so a casual build, or one on an unfamiliar toolchain whose newer diagnostics have not been chased down yet, still compiles. GCC and Clang have slightly different warning sets, so a clean local build on one is not proof the other is clean — that is what the separate CI legs exist to catch.

**MSVC is not in the gate yet.** `/W4` carries a pre-existing backlog (intentional integer narrowing, which GCC/Clang do not flag, plus some variable shadowing) that must be cleared before `/WX` can go on. That is tracked in issue #238; until it lands, `-DWERROR=on` is a no-op on MSVC and its `/W4` warnings stay visible in the log but non-fatal.

## The optional video backend (SDL3)

**Still no *required* dependency.** A host `Display` ( `src/host/display.h` ) is the seam any graphics board would draw into. That interface has two backends: a headless `NullDisplay` (draws into memory, shows nothing) that is always available, and an `SdlDisplay` (a real window) compiled **only when SDL3 is found**. So a plain `cmake -S . -B build` on a machine with no SDL3 still configures, builds, and passes every test — the display seam is simply headless, which is exactly what CI and a scripted run want.

To get a window, install SDL3 and reconfigure:

```
# macOS
brew install sdl3

# Debian/Ubuntu (needs a recent SDL3 package)
sudo apt install libsdl3-dev

# vcpkg (any OS)
vcpkg install sdl3

cmake -S . -B build           # reconfigure -- watch the status line
```

The configure step reports which backend it chose:

```
-- SDL3 found -- video boards enabled (windowed)
...or...
-- SDL3 not found -- video boards build headless (null display). Install SDL3 (see docs/devguide) for a
window.
```

CMake auto-detects with `find_package(SDL3 CONFIG)`; found → it compiles `src/host/display_sdl.cpp`, links `SDL3::SDL3`, and defines `SWTPCSIM_ENABLE_SDL`. Force a headless build even where SDL3 is installed with `-DSWTPCSIM_ENABLE_SDL=OFF`. That flag is what a macOS *universal* build needs, because a Homebrew SDL3 is single-arch and cannot link into an `x86_64;arm64` fat binary. **CI passes it nowhere. One leg — macOS — installs SDL3 from Homebrew and builds native `arm64`, so it is the only place `display_sdl.cpp` is compiled at all**, and the workflow fails that leg if it comes up headless. Linux and Windows take the not-found path above and build against the null display. Only `display_sdl.cpp` and the composition root (`src/main.cpp`) are macro-gated; the boards themselves `#include` no SDL and compile in every configuration.

## What actually got built

**Built and tested on Linux, macOS, and Windows.** The code is written to be portable — C++20, no dependencies, and every OS difference confined to `src/platform/` behind a header with zero conditionals — and that portability is now proven, not asserted: Linux (Ubuntu/GCC), macOS on Apple Silicon (and on Intel, built natively there rather than in CI), and Windows on MSVC all build and pass the suite. The Windows platform layer, once merely written, is field-proven.

**CI runs the suite on every push.** GitHub Actions builds and tests on all three platforms — Linux, macOS, and Windows are each a required check — so a regression on any of them shows up before it merges. The tests still run locally the same way, when someone types `ctest`.

**A merge does not rebuild what its PR already tested.** Merging an up-to-date PR makes a commit whose files are identical to the PR head CI just passed, so the run on `master` skips the build and is green with nothing to download. When the merge did bring in something new — PDFs the docs bot committed while the PR was open, a resolved conflict — only that difference is judged, so a PDF-only difference runs the single documentation leg. For the binaries of a merged change, fetch them from its PR.

Each of those jobs uploads the binary it built, so a green run leaves three executables on GitHub — including the two you cannot produce on your own machine. To fetch them:

```
tools/fetch-ci-binaries.sh          # newest CI run on the current branch
tools/fetch-ci-binaries.sh 42      # ...on PR 42
```

It **waits** for the run if it is still going and refuses to download from a red one, which makes it a reasonable last step before merging: three files means three platforms passed. They land in `./artifacts` (git-ignored, replaced on every fetch) with the executable bit restored, since the artifact zip does not carry POSIX modes. Nothing in the build or the tests reads from there — these are CI's binaries, kept for running or handing to someone, not a build output of this tree.

## The tests

```
ctest --test-dir build           # unit + acceptance. About 15 seconds.
ctest --test-dir build -L hw     # socket-hw + terminal-hw, against the real world.
```

The acceptance tests are not unit tests. They **boot period software on the whole machine through the real CLI** and check what lands on the terminal — FLEX 2.0 off a DC-4 floppy on the `swtpc`, MON680 in ROM on the `altair680`, and the KCACR audio cassette. Several ship with a **negative control**: the same script against a machine that ought to *fail*, marked `WILL_FAIL`. If a control ever passes, the test it guards was passing for the wrong reason and is worthless. That is the only reason to believe any of them.

The 6800 instruction set is covered by the `isa6800` unit suite ( `tests/test_isa6800.cpp` ), which checks the disassembler/assembler tables opcode by opcode.

## Catching memory bugs: **-DSANITIZE=on**

A crash on one platform only — typically Windows CI red, macOS and Linux green — is usually a use-after-free or use-after-scope that one allocator tolerates and another does not. No compiler warning fires for it.

`-DSANITIZE=on` finds it locally. It threads AddressSanitizer + UndefinedBehaviorSanitizer (just ASan on MSVC) through the whole binary, so the crash names its own `file:line` instead of you guessing from a block-buffered stack trace. Configure it in its **own build directory** — an instrumented binary is slower and is not what you ship:

```
cmake -S . -B build-asan -DSANITIZE=on
cmake --build build-asan --target swtpc_tests
ASAN_OPTIONS=detect_leaks=0 UBSAN_OPTIONS=print_stacktrace=1 ./build-asan/swtpc_tests
```

It is off by default and not in CI — CI catches a platform-only crash by running on that platform, and this is the tool you reach for once it does.

## The documentation is part of the build

Two documents come out of this tree:

| Target                                       | Is                                                                                                                                                                 |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Manual</b><br>( docs/manual/ )       | Ships in the package. <b>Self-contained</b> — it may not name a single file the reader does not have. No <code>src/</code> , no CMake, no <code>DESIGN.md</code> . |
| <b>Developer Guide</b><br>( docs/devguide/ ) | This. Repo only. Free to talk about the source, because you cannot write a board without it.                                                                       |

Some of the manual is **generated from the binary**, and this matters if you touch a board:

```
cmake --build build --target docs-reference
```

That rewrites `docs/manual/ref/*.md` — the command dictionary, the per-board parameter tables, the machine list — by walking `Board::properties()` and the `CommandDef` table. Those files are committed, and a **test ( docs-reference ) fails if they are stale**. So if you add a property, change a default, or reword a `HELP` string, run it and commit the result alongside your change.

This is not a docs chore bolted on at the end. A board's properties **are** its TOML schema; a hand-written parameter table would be a second schema, and the first draft of one in this very project got three of the memory board's eight defaults wrong. The reference is printed rather than retyped for the same reason the rest of the program has one source of truth for anything.

The PDFs need `pandoc` and any Chromium-based browser, and are built by `tools/build-docs.sh`. Neither tool is a build dependency, and neither goes anywhere near the simulator.

### Why a wrapped paragraph pastes as several lines (and why we don't "fix" it)

A recurring report (issue #246): copy a paragraph out of one of our PDFs and it pastes as several lines instead of one. This is a **reader** behavior, and the investigation is worth recording so it is not re-run from scratch.

**There are no hard line breaks in the paragraphs.** In the PDF content stream each wrapped line is painted at a new baseline position; there is no newline character anywhere. When a paragraph pastes as three lines, the reader is *inventing* those breaks from the vertical gaps at copy time. Nothing in the build put them there.

The only thing that tells a reader "the real text of this span is *this exact string*, ignore the geometry" is `ActualText`. Hand-built PDFs extracted with poppler make the picture exact:

| PDF                                                                         | Copies as                |
|-----------------------------------------------------------------------------|--------------------------|
| Wrapped paragraph, plain geometry                                           | 2 lines (break invented) |
| Wrapped paragraph inside a real <code>&lt;P&gt;</code> <b>structure tag</b> | still 2 lines            |
| Wrapped paragraph with paragraph-spanning <code>ActualText</code>           | <b>1 line</b>            |

The middle row is the one that closes off the easy fixes: the **structure/accessibility tree does not join wrapped lines** — only `ActualText` does. Our PDFs are already tagged ( `pdfinfo` → `Tagged: yes` ), and Chrome emits `ActualText` **only for ligatures** (the `fi` / `fl` spans), never for paragraphs. Every mainstream HTML/Markdown→PDF engine (WeasyPrint, Prince, wkhtmltopdf, Typst, LaTeX) conveys paragraphs through the structure tree, not through paragraph-level `ActualText` — so **swapping the PDF engine changes the producer name and nothing about the copy behavior**. Do not propose one as the fix.

Readers split on this: Word and Acrobat honor the structure tree, so the paragraph comes across whole; Preview and Chrome's built-in viewer fall back to pure geometry and invent the breaks. Worse, **macOS Sequoia's Preview/PDFKit regressed** — it stamps a paragraph break on *every* visual line regardless of the document, surviving even a text-only paste — so it would ignore `ActualText` too.

A custom `ActualText` post-pass is buildable — Chrome already brackets each paragraph in one marked-content span — but it is a real tool, not a shell snippet: the paragraph text is emitted as subsetting **hex glyph-IDs**, so each glyph has to be reverse-mapped through the font's `ToUnicode` CMap, with TJ kerning arrays and nested ligature/italic spans handled, then the stream recompressed. For a cosmetic gain that the reader in the original report (Sequoia Preview) would not even honor, it has not been worth doing.

The Markdown can't fix it either: wrapping is decided at render time from the column width, and there is no "soft wrap that copies as one line" a source file can encode. The only source-side lever is keeping the things people copy short enough **not to wrap** — which the commands already are (mono,  $\leq 79$  columns, in a column sized for 79). The case that wraps is prose. For copy-a-command / copy-a-prompt workflows, copy from the **Markdown**, which ships beside every PDF and is plain text; the PDF is the read-it version.

# Theory of operation

---

This chapter is about what is actually going on inside the machine. It assumes you have the source, and it names files: `src/core/bus.h`, `src/core/board.h`, `src/core/value.h`, `src/core/machine.h`, and `DESIGN.md`, which is the document all of this was argued out in.

The next chapter builds a board — a lamp latch on port `FF`. Everything here is what that board is standing on.

## The whole architecture, in one sentence

**Boards respond to bus cycles. The CPU originates them.**

That is not a slogan. It is the shape of the code, and everything else in this chapter is a consequence of it.

The two concepts are separate types, and they live together in `src/core/bus.h` rather than with the CPU:

```
class BusMaster {
public:
    virtual StepResult step(Bus&) = 0;
};
```

A CPU board is **both**: `Cpu6800Board : public Board, public BusMaster`. It is a board you can pull out with your hand ( `BOARDS REMOVE cpu0` works, and a machine with no processor in it is a real machine), and it is also the thing that drives cycles onto the backplane.

**Making the CPU a `BusMaster` — not a bus special case — is what keeps the model clean.** It is how the monitor *finds* and *steps* the processor generically, and it is the door a DMA card would walk through unchanged: a `Board` that *becomes* a `BusMaster` when it is granted the bus, driving the very same cycles through the very same interface the CPU uses. **DMA would never be a special path bolted onto the bus.** (No 6800 machine here masters the bus from anything but the CPU; the framework's S-100 `pHOLD / pHLDA` DMA arbitration was removed — `DESIGN.md` §4.5 — but the door it used is still open.)

It is also why `BREAK MEM W 0100` catches a write no matter what drove it. A cycle is a cycle; nothing on the backplane records who originated it. There is deliberately **no `origin` field** on `BusCycle`, and `src/core/bus.h` says why: a real backplane cycle carries no such tag, which is exactly why an operator's `DEPOSIT` is indistinguishable from a CPU write, and why a real ROM ignores both.

## The bus carries signals. It does not invent behavior.

The second rule, and the one the bus header exists to enforce:

The bus arbitrates no overlay, vectors no interrupt, knows no bank, and has never heard of ROM.

It picks no winner between two cards. It does not hand the CPU an interrupt vector. It does not route reads to one board and writes to another. Every one of those lives in a board.

When you are tempted to add `if (board is a ROM)` to `Bus`, **you have found a bug in your board instead**. There is exactly one thing the bus does that no board does, and §4.6.1 of DESIGN.md pins it down: it supplies `0xFF` when nobody answered. That is the entire bus/board overlap, and it stays that size.

**This is the SS-50 bus, and the 6800 has no I/O space.** The `BusCycle` above carries a cycle type, a 16-bit address, and a data byte — and the type is only ever `MemRead` or `MemWrite`, because the 6800 reaches every device through memory, with the same loads and stores it uses for RAM. There is no IN/OUT, no port table, no separate I/O strobe. A device is just an address that a board answers, and the SS-30 I/O bus is a window of such addresses ( `$8000 – $801F` ) hanging off the SS-50 motherboard. **DESIGN.md §4.0** argues out where that line is drawn.

## A bus cycle, end to end

```
enum class Cycle { MemRead, MemWrite };

struct BusCycle {
    Cycle type = Cycle::MemRead;
    uint16_t addr = 0;    // the address on the bus
    uint8_t data = 0;

    bool isWrite() const { return type == Cycle::MemWrite; }
};
```

Two cycle types — a read and a write, both to memory, because on the 6800 there is no other kind.

`data` is valid on a write. **On a read it is zero while the cycle is in flight** — nobody has driven the bus yet when `decodes()` and `read()` are asked, so there is nothing honest to put there — and it is **back-**

**filled** with the byte that came back (a board's, or the floating bus's `0xFF`) before the observers see it.

`Bus::settle()`.

The bus runs each cycle in two passes:

| Pass | What it does                                                                                                                       |
|------|------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Ask who <code>decodes()</code> this cycle. Exactly one should answer. Nobody → <code>0xFF</code> .                                 |
| 2    | <code>settle()</code> : hand the completed cycle to every observer watching from outside the backplane (the debugger, the tracer). |

Running a decode and letting the watchers see the result. **No decisions.**

## The decode is cached, because on real hardware it is wired

A board's address decoder is combinational logic — a PAL, a row of gates — wired to the address lines and to the  $R/\bar{W}$  line that says which way the byte is going. It does not *answer a question* once per cycle. It **settles**, and it only changes when something latches: a bank strap, an enable/disable, a board pulled from the backplane.

So the bus asks the same questions of the same boards in slot order, and asks them **once**, storing the answer in two tables:

```
struct Slot {
    Board* who = nullptr; // the single board that drives. null: nobody -> floats 0xFF
    bool slow = false;    // more than one driver. Contention: take the exact path.
};

Slot memRead_[256], memWrite_[256];
```

One entry per **256-byte page**, per direction. A decode is now a table lookup, not a walk over every board in the machine.

The board still owns the entire decision. The bus still invents nothing. It just stopped asking sixty-five thousand times a second — and stopped asking boards questions they are not wired for. A board that only decodes a write — a plain output latch — used to be asked to decode every read too, and its first act was to throw the question away. **It has no connection to the read direction. It is not in that conversation.**

Three consequences follow, and all three are things you can get wrong.

**`decodes()` must be pure**

```
virtual bool decodes(const BusCycle&) const { return false; }
```

**Same board state, same cycle → same answer, and no side effects.** `decodes()` is combinational. The bus may call it more than once per cycle — inside its own decode and again when it re-derives the cache — and then it *caches the result*.

A `decodes()` with a side effect in it is a bug that will not surface for a month. It will fire the right number of times on the day you write it and the wrong number of times forever after, because the number of times the bus asks is an implementation detail of a cache. **Latch nothing here** — a `decodes()` has no business changing anything about the board.

## If your decode changes, say so

```
void decodeChanged(); // "my decode just changed" -- sets a dirty flag on the bus
```

A bank strap moved. A chip came out of a socket. The board went `enabled = false`. **Call `decodeChanged()`**. Forget, and the tables go stale and the machine lies quietly, which is the worst failure mode there is.

You mostly get this for free: `setProperty()` — the one path by which any property is *ever* set, from `SET`, from `TOML`, from `BOARDS ADD`, from `MCP` — calls `configChanged()` on the board after every successful set, and the default `configChanged()` calls `decodeChanged()` and `intChanged()`. You call it by hand for changes that do not come through a property: a guest store that moves a bank strap, a boot ROM switching itself out.

And it is not left to trust. `Bus::setVerify(true)` re-derives the decode the slow way on every single cycle and screams the moment it disagrees with the table. The unit suites run with it on permanently. **It is a proof, not a path** — it is slower than the code it replaced, and that is fine.

## The bus routes by CYCLE TYPE, not just by address

Two tables, not one. That is not an optimization detail — it is the model:

**A board is wired to the address lines and to  $R/\bar{W}$ .** A ROM famously does not decode a write *at all*; it does not reject the write, or ignore it, or log it — it never answers the cycle. So "who answers here" has a **different answer for a read than for a write**, and that falls out of the model rather than being bolted onto it.

The consequence you will use immediately: **one address can mean two different things by direction, with no contention whatsoever.** That is not a trick. It is what the 6850 ACIA does on the console board: a *read* of `$8004` is its status register and a *write* of `$8004` is its control register — one address, two registers, told apart by  $R/\bar{W}$ . `src/boards/swtpc-mps.cpp` decodes both directions of the slot and its `read() / write()` hand back the right register for each.

The other half of the same coin is a board that answers only *one* direction. An output latch — the next chapter's lamp board — decodes the **write** at its address and leaves the read alone:

```
bool decodes(const BusCycle& c) const override {
    return enabled_ && c.type == Cycle::MemWrite && c.addr == addr_;
}
```

A load from that address is **not the latch's** — it goes unclaimed and floats to `0xFF`, which is precisely what a real output-only port does. So the read at that address is free space, and that is where the next chapter's lamp board lives.

### `decodeIsPageUniform()` — when a card decodes a low address line

```
virtual bool decodeIsPageUniform() const { return true; }
```

Memory decode is cached one entry per 256-byte page, and that is a **contract on `decodes()`**. Nearly every board can keep it: memory decoding comes off the *high* address lines, and a board selected at 1K or 4K granularity answers a whole page or none of it.

A board that decodes only *part* of a page says `false` to `decodeIsPageUniform()`. The MP-S serial board is the real one — it answers at `8004 - 8007`, four addresses inside page `80`, and nothing else — so some addresses of that page are its and most are not. Say `false` and the bus probes every address of any page whose answer is not uniform, serving it by the exact, uncached two-pass path. **You lose nothing but the cache, and only on the pages you actually touch.**

### The cycle stream — the debugger observes from outside

Every cycle is on the backplane. That is what a backplane IS — the address bus is not addressed *to* anyone, it is simply present. The debugger watches that same stream from **outside** the backplane through `Bus::observe()`:

```
using Observer = std::function<void(const BusCycle&)>;
int observe(Observer fn); // returns a handle; unobserve(handle) removes it
```

An observer is called **once per cycle, after it completes**, with `data` back-filled. That one hook is the entire implementation of `BREAK MEM`, `TRACE` and `HISTORY` — they are questions about bus cycles, answered at the bus, so they cost the cores nothing and the 6809 inherited them for free. The bus is not *notifying* anyone: the cycle was on the backplane the whole time, and an observer is just a probe clipped onto the wires.

(A `BREAK MEM` must stop the machine *before* the access, with the PC still on the instruction — which an after-the-fact observer cannot do — so that one case rides a separate hook, the pre-access veto,

`Bus::setPreAccessVeto` . `TRACE` and `HISTORY` only record, so they ride the observer.)

## `peek()` — look without touching

```
virtual bool peek(uint16_t addr, uint8_t& out) const { return false; }
```

A `read()` may CONSUME. A read from a 6850's data register takes the byte and the guest never sees it again. So `DISASM` , `TRACE` and the debugger's register display are built on `peek()` , never on a read — a disassembler that ate the console's input the first time someone disassembled a page with a 6850 mapped into it would be a debugger you could not trust.

`peek()` runs the same decode, so a board that is disabled or banked away is invisible to it exactly as it is to a real read. It just never strobes anybody.

A board that cannot answer without side effects returns `false` , **and that is an honest answer, not a failure**: the byte on a real bus is only defined *during* a cycle. The caller shows `FF` , which is what the bus would have floated to.

## The floating bus — one rule, three consequences

If no board drives the bus, it floats high. Every read of an unmapped address or port returns `0xFF` . Writes go nowhere.

One rule. Two things that would otherwise each need a special case fall straight out of it:

| Situation           | What happens            | Why it matters                                                                                                                                       |
|---------------------|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unpopulated memory  | reads <code>0xFF</code> | Period software <b>sizes memory by reading</b> . Return <code>0x00</code> and every machine looks like it has 64K, and software builds itself wrong. |
| An empty SS-30 slot | reads <code>0xFF</code> | A guest probing for a device that is not there gets the answer real hardware gives it.                                                               |

Model the floating bus honestly once and both are free. Fake either one and you will fake the other differently.

**Therefore no board may ever manufacture `0xFF`** , and in particular no board may seed its own store with it. A RAM chip does not power up holding `FF` ; it powers up holding whatever it feels like, which is what `fill = random` is for. Seed a board's store with `FF` and `DUMP` shows `FF` for a board whose RAM is fine, `FF` for a board whose RAM was never filled, and `FF` for a board that **isn't in the machine**. One

symptom, three causes. The moment a board can produce `FF`, the only signal the bus has stops being a signal. `tests/test_boundary.cpp` enforces it.

## Interrupts

One wire's worth of interface, and it obeys the same rules as `decodes()`.

```
virtual bool assertsInt() const { return false; } // pulling the IRQ line?
void        intChanged();                       // "my pin may have moved"
```

The 6800 has one maskable interrupt line, **IRQ**, plus a non-maskable **NMI** and the software **SWI**. All three vector through fixed locations at the top of memory (IRQ `$FFF8`, SWI `$FFFA`, NMI `$FFFC`). A board that wants to interrupt pulls the IRQ line and holds it; the CPU reads its handler address out of `$FFF8` and takes it when the **I** mask is clear. The bus carries a level and never invents a vector — there is no acknowledge cycle and no card that drives the data bus during one. (That was the S-100 lineage; see the heritage note in DESIGN.md §4.4.)

### An interrupt is a LEVEL, not an event

A UART with a character waiting and its receive interrupt enabled says `true`, **and keeps saying true until the guest reads the character**. There is no queue, so a board cannot "lose" an interrupt — there was never a queue to lose it from.

`assertsInt()` is **combinational and pure**, exactly like `decodes()`. It reports the settled state of a pin, computed from the state of the chip and nothing else. It does not advance a receiver, take a byte off a line, or do any work the guest has not paid for.

**It used to do all three**, and there was a whole section of DESIGN.md defending it. The 6850 has to notice a character has finished arriving, which happens on the chip's own clock with no help from the CPU — and being asked `assertsInt()` was the only thing that ever woke the card up. **The poll was serving as the card's clock**. That work moved to where it belongs: a `clock` deadline the card sets for itself, and `pump()`. Read §4.4.1 of DESIGN.md before you are tempted to do work inside it.

### The bus does not poll. It keeps the wire.

`Bus::intPending()` used to walk the backplane and ask every card `assertsInt()`, **once per instruction** — sixty million times a second, to compute a boolean that changes maybe a thousand times a second. It was the single largest per-instruction cost left in the simulator once the decode was cached, and it *grew with every card you added*.

Now the board **pulls the pin** and the bus keeps a running wire-OR (`intCount_`). Reading the IRQ line is one integer test, flat in the number of cards.

The two hard-won rules turn out to be one rule:

|                | combinational, pure       | "it moved"                   | what the bus keeps |
|----------------|---------------------------|------------------------------|--------------------|
| address decode | <code>decodes()</code>    | <code>decodeChanged()</code> | a page table       |
| interrupt      | <code>assertsInt()</code> | <code>intChanged()</code>    | a wire-OR count    |

**A board's outputs are pure functions of its state. When its state changes, it says so. The bus caches the rest.**

Call `intChanged()` after anything that could move the pin: a register written, a character taken off the line, a deadline coming due. A spurious call costs a virtual call. **A missing one hangs the guest forever**, waiting for an interrupt that already happened — and it presents as *"the emulator locks up sometimes"*, which is worth a week of anyone's life.

So it is not left to trust either. `Bus::setVerify(true)` re-derives the wire from every board's `assertsInt()` on every instruction, and aborts the moment a board disagrees with the cache. A stale interrupt wire hangs the guest exactly this way, and is otherwise near-impossible to see.

## Memory and ROM

A memory card is a **list of regions**, not an address range: RAM here, a ROM socket there, an empty socket between them. One card can occupy two separate ranges, because a real one does. `SHOW BUS MAP` is per-range, not per-board, for that reason.

**An empty socket decodes nothing.** It does not read as zero — it floats to `FF`, like anything else nobody drives.

**A ROM region never answers a write.** Its `decodes()` returns `false` for a `MemWrite`, so a guest store to ROM lands nowhere — the write half of the floating bus — and the bus needed no rule about ROM to make it happen. Reads and writes decode separately (the two `Slot` tables), which is what makes that fall out for free.

**Heritage note — PHANTOM\*.** The S-100 machines the framework came from had a `PHANTOM*` line (pin 67): a boot ROM pulled it, memory boards strapped to honour it took *themselves* off the bus, and the overlay was **emergent** — the ROM the only board still answering because the RAM switched itself off, the bus picking no winner. No 6800 machine here shadows memory (the SWTPC and Altair-680 maps place ROM and RAM in disjoint ranges), so `PHANTOM*`, the `assertsPhantom` hook and the `honors_phantom` strap were all removed. The lesson it taught survives: an overlay is two ordinary board behaviours, never a bus special case picking a winner.

### `rawRead` / `rawWrite` / `rawSize` — the PROM burner

```
virtual size_t rawSize() const { return 0; }
virtual uint8_t rawRead(size_t) const { return 0xFF; }
virtual bool rawWrite(size_t, uint8_t) { return false; }
```

Straight into the card's backing store, **bypassing decode entirely**. Offsets are board-local, and the store may be far larger than 64K — a banked card's bank 3 simply is offset `0x30000`.

**This is the PROM burner, and that is not a metaphor.** It is how the operator writes a ROM the guest cannot, because **burning a PROM is not a bus operation on real hardware either. You pull the chip.**

Model it as a bus write instead and the bus would have to know *who originated a cycle* — which a real backplane cannot know, and which no board should ever have to ask.

## Reflection is the keystone

```
virtual std::vector<Property> properties() = 0;

struct Property {
    std::string name;           // "baud", "fill", "clock_hz"
    std::string help;          // one line, shown by SHOW
    Kind kind = Kind::Int;      // Int | Bool | Str | Enum

    std::vector<std::string> choices; // Kind::Enum -- also feeds tab completion
    long long min = 0, max = 0;      // Kind::Int; min==max means unbounded
    int radix = 10;              // 16 for addresses, so SHOW reads right
    std::string unit;           // "Hz", "bytes" -- display only

    std::function<Value()> get;
    std::function<bool(const Value&, std::string& err)> set;
};
```

`SET`, `SHOW`, the TOML loader, `CONFIG SAVE`, the MCP tool schemas, tab completion and the manual's **generated board reference** are all written **once**, against this. They know nothing board-specific.

**There is no second schema anywhere.** A board's TOML keys *are* its properties. A board added next year is configurable, scriptable, agent-drivable and documented **the day it lands**, and none of those six consumers changes a line.

The cost of that is that a property has to carry enough metadata to validate, render and describe itself. That is the whole trick, and it is why `radix` is there: `PORT=10` is port `0x10` and `BAUD=9600` is nine thousand six hundred, because the property said so.

Three things to get right when you write one.

**A property with no setter is read-only.** Live pin state, a derived value, a lamp. Leave `set` empty — **and that absence is the only signal any consumer has**. A setter that always refuses would stop a `SET`, and simultaneously fool `SHOW`, `CONFIG SAVE`, the MCP schema and the generated docs, all four of which read the *presence* of the function. Do not write one.

**There is no config-time-only property.** Every property can be set, always. You can only type at the prompt when the machine is stopped — that is what the STOP key does, returning you to the monitor, and there is no moment at which a `SET` races a running CPU. And on real hardware the gate would be a fiction anyway: a board being worked on sits on an **extender**, out where you can reach it, and its jumpers get moved with the power on.

**`unitProperties()` is for a real sub-thing with its own settings.**

```
virtual std::vector<Property> unitProperties(const std::string& unit) { return {}; }
```

A board with two independent 6850s carries **two channels** with their own crystals' worth of jumpers — independent baud rates, independent transforms — so they are units, and `SET <board>:a BAUD=9600` reaches one of them. Folding them into `properties()` as `a_baud / b_baud` would work for two channels and fall apart on the first board with eight ports. Most boards return `{}` here, and that is fine.

A unit is a **name and a kind** (`UnitDef`, `UnitKind::`{`Disk`, `Rom`, `Serial`, `Tape`, `Cpu`}), never an index — so `MOUNT dj:drive0`, and mounting a disk image onto a serial port is an *error with a sentence*, not undefined behaviour that half-works. `units()` is the only list; `SHOW`, `MOUNT`, `CONNECT`, the MCP schemas and tab completion all read it, so they cannot disagree about what exists.

## Reset is two different events

```
enum class Reset {  
    PowerOn, // POC* -- power-on clear. Nothing in software can assert it; only the power supply.  
    Bus      // RESET* -- the RESET line. Warm.  
};  
  
virtual void reset(Reset) {}  
virtual void power() {}
```

Mixing these up is the classic source of *"works from power-on but not from the reset button."*

**Neither reset clears memory. Only removing power does.** That is not a nuance, it is the rule, and the memory array is the proof: **a RAM chip has no POC\* pin.** Its contents are indeterminate at power-up because *the chips just powered up*, not because a signal arrived. A `Reset::Bus` leaves memory intact and leaves media mounted and streams connected.

`power()` is the only thing that loses RAM and re-reads ROM images.

**What each signal does is a fact about your board, and it comes from the manual.** It is tempting to scrub the card clean on `RESET*` because it feels safe, and it is exactly backwards: **a card that resets more of itself than the reset line physically reaches is inventing a machine nobody built, and the invention is destructive.**

A 6850 board proves it. The **MC6850 has no RESET pin** — 24 pins, and RESET is not among them — so `RESET*` reaches that board's address decoding and *nothing else*. This tree used to reset the ACIA anyway, throwing away the guest's word format and interrupt enables and eating a byte out of the receive register, on a board where a real bus reset would have preserved all of it.

The memory board is the model to copy: **it clears its bank latch on either reset and touches not one byte of RAM.**

Your board's `.md` must say concretely what each of the two does to it. That is a gate, not a nicety (DESIGN.md §14).

## Lifecycle, time, and the host

`pump()` — the one door to the outside world

```
virtual void pump() {}
```

Give the host a turn: accept a socket connection, drain a keyboard. It is called **once per time slice by the run loop, and NEVER from inside a bus cycle**.

**That seam is what keeps a board pure.** `read()` and `write()` are pure computation over the card's state; anything that has to *talk to the outside world* happens here, at a known point in emulated time. That is what would let a recorded session replay identically instead of depending on when the host scheduler happened to deliver a packet.

Boards must **never** touch a socket, a file handle or `termios` directly. Everything they need from the host comes through the services in `src/host/` and `src/platform/`.

## **Clock** — emulated time

Time is measured in **cycles**, never milliseconds, and it advances **only when the CPU retires an instruction, by exactly the count the CPU reported** (`StepResult::cycles`). `Machine::clock` is the one clock; a board is handed it by `attachClock()` when it goes into the backplane.

Nothing else in the simulator may call `std::chrono::now()`. That is what makes the guest's sense of time and a card's sense of time *the same sense of time* — a UART's idea of when a character has finished going out is derived from the very instruction stream the guest is timing it against, so the two cannot drift.

A card with nothing time-dependent on it never looks at the clock, and most don't. A UART absolutely does: **TDRE is a deadline, not a flag**. Two facilities, and they answer different questions:

- `Clock::at() / after()` — a *deadline*: something emulated time already knows is coming. A character finishing transmission. A sector arriving under the head.
- `pump()` — a *keystroke*, which is not in emulated time at all and which nothing could have scheduled.

The 6850 needs both, in the same function: `pump()` takes the byte off the line, and if the line has not yet had time to deliver it, sets a deadline for when it will.

**The crystal is on the CPU card**, so `clock_hz` is that board's property. There is no machine clock and there must never be one again — a machine-level copy would be a second place to say one thing, and the day the two disagreed the machine would run at whichever was written last.

## **drainLog()** — what the card wants said out loud

```
virtual std::vector<std::string> drainLog() { return {}; }
```

A bank select it could not decode. A ROM that failed to load. A sector whose checksum did not match. Drained by the monitor after every command and after every run, and cleared by the draining; MCP returns the same strings as structured data.

It is on `Board`, and it is virtual, because it used to be a `dynamic_cast<MemoryBoard*>` in `Machine::drainBoardLog()` — which meant a disk controller with something to say about a bad sector **had no way to say it**. A general facility with one card's name compiled into it is a bug, every time.

## A chip is not a board

`src/chips/` holds the parts that get soldered onto more than one board: `mc6850.h` (the MP-S console and the 680b onboard/Universal I/O), `uart1602.h` (the COM2502 — the 680b KCACR cassette), `wd17xx.h` (the DC-4's FD1791).

**Each chip is modelled from its DATA SHEET. Each board is modelled from its MANUAL.** A chip built instead from the one BIOS that happens to drive it implements exactly the subset that BIOS touches and quietly gets the rest wrong — and it looks finished while doing it.

**A chip knows nothing about the bus.** It has a clock, some pins, and (if it moves bytes) a `ByteStream`. What it talks to is an **interface, never a file**: `Wd1791` has a `FloppyDrive` — STEP, DIRC, TR00, WPRT, READY, IP, with a drive on the far end. It owns no image, no geometry and no host file, and it has **no drive-select and no side-select pin**, because the FD179x hasn't got either. Which drive those pins reach is a **latch on the board**. That absence is the chip/board seam stated in silicon.

**And the seam is not "the chip does the work and the board forwards to it."** A board whose status word is **inverted** and one whose is not do not share a shared UART class with a `bool invert` on it — that is *precisely* the bug `src/chips/` exists to prevent, so such boards share **no code**, on purpose. What permits sharing is being **the same part**, not filling the same role.

**Where the seam falls is a fact about the chip, not a house style.** The chip is what the data sheet describes, at its pins, in true sense. Everything between those pins and the bus — the inverting buffers, which bit each signal lands on, the interrupt-enable flip-flops that live in a *different IC*, the port decode — is the **board's**, and it stays on the board.

## Where this leaves you

You now know what a board is: a thing that is asked two pure questions (`decodes`, `assertsInt`), that is *told* two things (`read`, `write`), that describes itself (`properties`), and that gets a turn to talk to the host at one known point in emulated time (`pump`). The debugger watches the same cycles from outside the backplane (`Bus::observe`), never through the board. Everything else on the vtable is a detail of one of those.

You also know the traps: a decode with a side effect in it, a decode that changed without saying so, a missing `intChanged()`, a card that resets more of itself than the real one does, and a board that manufactures its own `0xFF`.

**That is enough to write a board. The next chapter writes one** — a lamp latch that decodes the *store* at an empty SS-30 address, leaving the load there to float, which is the whole lesson made concrete.

# Writing a board

---

We are going to build a board. A real one — it plugs into the backplane, it decodes a bus cycle, it holds state, it has a setting you can put in a machine file, and when we are done the monitor will know about it, `CONFIG SAVE` will write it out, and an AI assistant driving the machine over MCP will be able to configure it. None of that last part will cost us a line of code.

The board is **eight lamps and a latch**. A store to its address lights them. That is the whole thing.

The finished source is in the tree at `examples/boards/lamp/lamp.h`, and `tests/test_lamp.cpp` drives it on a real bus — so it compiles and it is tested, which is not something you can say about most tutorial code. Read along, or write it yourself.

## Why an address, and why only the write

The 6800 has no IN/OUT space. It reaches every device the same way it reaches RAM: by **address**, with the ordinary load and store instructions. So our board does not own a "port" — it owns an address, and it lives on the **SS-30 I/O bus**, the eight-slot window at `$8000 – $801F` where the console and the disk controller already sit (slot  $N$  is based at `$8000 + 4 · N`). Slot 4, `$8010`, is empty in the stock `swtpc` machine. Ask it:

```
swtpcsim> WHO 8010
8010 read  nobody -- floats to FF
8010 write nobody -- floats to FF (a write here is simply gone)
```

Nobody, either direction — so the lamp can have it. But watch which direction it takes. A **store** lights the lamps; a **load** has nothing to answer — there is no "what are the lamps?" register on the real thing, just eight pins driving eight LEDs. So the board claims the *write* at `$8010` and leaves the *read* alone, and a load from `$8010` floats to `0xFF` like any unclaimed address.

That is not a shortcut. It is how 6800 I/O actually works, and the 6850 ACIA in the next slot proves it: its **status** register is a read at `$8004` and its **control** register is a write at the *same* `$8004`. One address, two registers, told apart by direction. The bus can do that because it routes each cycle by its **type** — `Cycle::MemRead` and `Cycle::MemWrite` are different cycles, decoded separately (`src/core/bus.h`), for the same reason the real backplane carries  $R/\bar{W}$  alongside the address lines.

**The bus routes by cycle type, not just by address.**

Keep that in mind while writing `decodes()` — a board that answered the *read* at `$8010` would be claiming a byte it has no answer for.

## 1. The board

`examples/boards/lamp/lamp.h`. A board is a class that inherits `Board` (`src/core/board.h`) and answers some questions.

### Its name

```
std::string type() const override { return "lamp"; }
```

This is the word a machine file uses: `type = "lamp"`. It is the **chip or the common name**, never a catalog number — nobody ever asked for an MP-A board, they asked for a 6800.

### What it decodes

```
bool decodes(const BusCycle& c) const override {  
    if (!enabled_) return false;           // a board switched off drives nothing  
    if (c.type != Cycle::MemWrite) return false; // a store lights them; a load is not ours  
    return c.addr == addr_;  
}
```

That middle line is the whole lesson of this chapter. Delete it and the board answers the *load* at `$8010` too — a read that should float to `0xFF` now returns the latch, and any program that probes the empty slot gets a lie instead of the floating bus.

One more line earns its place, and it is easy to leave out:

```
bool decodeIsPageUniform() const override { return false; } // one address, not the page
```

The bus caches decode a page at a time — 256 bytes — because most boards answer a whole page uniformly and it would be wasteful to ask 256 times. Our board answers *one* address inside page `$80`, a page it shares with the console and the disk controller, so it must tell the bus to ask per address. Forget this and the cache smears one answer across the page and boards collide. (The `mps` and `dc4` boards say the same thing, for the same reason.)

Two rules about `decodes()`, and they are not negotiable:

- **It must be pure and combinational.** It answers "*if this cycle happened, would I drive the bus?*" and it does nothing else. No side effects, no counters, no latching.
- **The bus caches the answer.** It keeps one cached answer per page (and, for a board like ours that says its decode is not page-uniform, per address), so a decode is a table lookup rather than a walk

over every board in the machine. Which is why a `decodes()` with a side effect in it is a bug that will not show up for a month: it does not get called when you think it does.

If a board's decode ever *changes*, it must say so — `decodeChanged()`. (You will see below that we get that for free.)

## What it does with the cycle

```
void write(const BusCycle& c) override { latch_ = c.data; }
```

We claimed the cycle, so the byte is ours.

We never override `read()`. We never say yes to a read, so we are never asked one — and a load from `$8010` floats to `0xFF`, which is exactly what an output-only board does.

## Power and reset

```
void reset(Reset) override { latch_ = 0; }  
void power() override      { latch_ = 0; }
```

These are **two different events** and a board is entitled to treat them differently. `Reset::Bus` is the RESET\* line — the reset the machine asserts when you press it. `Reset::PowerOn` is the power coming up, and it is the only thing that loses RAM. For our lamps they mean the same thing: the lights go out. For a memory board they emphatically do not.

## Its state — SNAPSHOT and RESTORE

A board writes its **runtime state** so `SNAPSHOT` can save it and `RESTORE` can put it back (`DESIGN.md §13`). Chain to the base — it handles `enabled_` — then read and write your own fields in the same order:

```
void serialize(StateWriter& w) const override {  
    Board::serialize(w);  
    w.u8(latch_);  
}  
void deserialize(StateReader& r) override {  
    Board::deserialize(r);  
    latch_ = r.u8();  
}
```

`StateWriter/StateReader` (`core/statefile.h`) are explicit and little-endian — `u8`, `u16`, `u32`, `u64`, `boolean`, `str`, `blob`, and `raw` for a fixed array — so a snapshot reads back byte-for-byte on every target. Three rules decide what goes in:

- **Write runtime state, not configuration.** A snapshot is RESTORED into a machine built from the same machine file, so your straps (the port, a baud rate, a jumper) are already correct — writing them would be a second copy that could disagree. Write the registers, latches, RAM and in-flight buffers that a running machine accumulates.
- **Do not write a host handle.** A `ByteStream`, a disk image, the `Display` — those are re-opened from the (unchanged) config. But bytes that are **not on the host yet** — an uncommitted write buffer, a tape's head position — *are* your state and *do* travel.
- **Never write a `Clock::Handle`.** The event queue does not survive a snapshot. If your board sets a deadline, **re-arm it in `deserialize()`** from the state you just read — call the same `refresh()` / `arm...`() you already call when a jumper moves. Store the deadline as an absolute cycle if you need the timing back; it stays valid because the clock's time travels with it.

## Its settings — and this is the part that pays

```
std::vector<Property> properties() override {
    std::vector<Property> p;
    {
        Property x;
        x.name = "addr";
        x.help = "the SS-30 address this board latches. Write-only -- a load here is not ours";
        x.kind = Kind::Int;
        x.radix = 16;           // ON THE WIRE -> HEX. An address is a thing the CPU sees.
        x.min = 0x8000;
        x.max = 0x801F;
        x.get = [this] { return Value::ofInt(addr_); };
        x.set = [this](const Value& v, std::string&) {
            addr_ = (uint16_t)v.i();
            return true;
        };
        p.push_back(std::move(x));
    }
    {
        Property x;
        x.name = "lamps";
        x.help = "what the guest last wrote -- the eight LEDs";
        x.kind = Kind::Int;
        x.radix = 16;
        x.get = [this] { return Value::ofInt(latch_); };
        // NO SETTER.
        p.push_back(std::move(x));
    }
    return p;
}
```

**This vector is the entire configuration layer.** There is no schema file, no parser, no registration call, and nowhere else to declare anything. `SET`, `SHOW`, the TOML loader, `CONFIG SAVE`, the MCP tool schemas, tab completion, and the User Manual's generated board reference are all written **once**, against this, and know nothing about any particular board.

Two details worth stealing:

- `radix = 16`, because an address is a thing the processor can see. On the wire → hex; never on the wire → decimal. Get this wrong and `addr = 8010` in a machine file quietly means eight thousand and ten instead of the address `$8010`. **It declares the class, not a display preference** — the one `radix` both reads bare digits and prints them, an operator can force any base of their own (`0x`, `0o`, `0b`, `#`) whatever you chose, and `SET CONSOLE base=octal` does not reach it: that switches how the *monitor* prints addresses and bytes, while `SHOW` prints every property in its own radix. So pick it by which side of the wire the number lives on, and never because one base reads better.
- **lamps has no setter**, and that absence is the signal. `SHOW` prints `(read-only)`, `CONFIG SAVE` leaves it out of the file, and the manual's reference marks it. A setter that merely *refused* would stop a `SET` and fool all three at once — which is a mistake that was in this codebase, on the memory board, until this manual went looking.
- **We never call `decodeChanged()`** in the `port` setter. The property layer calls it for us after any successful set, precisely so that a board author cannot forget.

## What it tells the operator

```
std::vector<MapEntry> memMap() const override {  
    return {{addr_, addr_, "write", "lamp latch -- D0..D7 (write-only)}};  
}
```

This is what `BOARDS`, `SHOW BUS MAP` and `WHO` print. **It is documentation, not decode** — the bus never consults it. A board whose `memMap()` disagreed with its `decodes()` would work perfectly and lie to you, which is worse than a board that does not work.

## 2. Put it in the registry

Two lines and an include, in `src/boards/registry.cpp`:

```
#include "../examples/boards/lamp/lamp.h"

// ...in boardTypes():
{"lamp", "Eight-LED output latch (the Developer Guide's example)",
  "A write-only latch: a store lights eight LEDs. The Developer Guide builds this"},

// ...in makeBoard():
if (type == "lamp") return std::make_unique<LampBoard>();
```

The row is the type name, a one-line summary for the `SHOW BOARDS` catalog (keep it short enough to fit the column -- a test checks every row fits 78 columns), and the full description that `SHOW BOARD <type>` prints.

That is all. `registry.h` promises *"adding a board type is one line here and nothing anywhere else"*, and it means it.

(Our board is a header, so it needs no entry in `CMakeLists.txt`. A real one — with a `.cpp` — adds its source to the `swtpe_core` library alongside the others.)

### 3. Build it, and watch what you get for free

```
$ cmake --build build -j
```

The board is now in the catalogue. `SHOW BOARDS` lists every type with its one-line summary; name one to see its settings, their help text and their legal values -- and nobody wrote a line of code to put it there:

```
swt pcsim> SHOW BOARDS
TYPE      DESCRIPTION
-----
...
lamp      Eight-LED output latch (the Developer Guide's example)
...

SHOW BOARD <type> for a board's properties (add UNITS for just the units)

swt pcsim> SHOW BOARD lamp
lamp      A write-only latch: a store lights eight LEDs. The Developer Guide
          builds this

PROPERTY  HELP
-----
addr      the SS-30 address this board latches. Write-only -- a load here is
          not ours
          values: 0x8000 .. 0x801F
lamps     what the guest last wrote -- the eight LEDs
```

Fit one, and ask the machine the same question we asked at the start:

```
swt pcsim> BOARDS ADD lamp lamp0
lamp0: lamp added

swt pcsim> WHO 8010
8010 read  nobody -- floats to FF
8010 write lamp0
```

It said `nobody` on the write an hour ago. Now light the lamps — `DEPOSIT` runs a store cycle and says nothing, because a store says nothing; the byte shows up in `SHOW`:

```
swt pcsim> DEPOSIT 8010 55

swt pcsim> SHOW lamp0
lamp0 (lamp)

property  value      legal
addr      0x8010      0x8000..0x801F
lamps     0x55      (read-only)
```

`DEPOSIT 8010 55` ran a **real store cycle on a real bus** — the same path a CPU's own store takes, because there is only one. The board decoded it, latched it, and `SHOW` read it back out of the reflection layer.

`lamps` is marked read-only, and it worked that out from the missing setter.

And the *read* at that address is still nobody's — `WHO 8010` said so up above, and a load there floats to `0xFF`. One address, two directions: the write is the lamp's and the read is nobody's, with no contention. **Because the bus routes by cycle type.**

## 4. Put it in a machine

```
[machine]
name = "lamps"
base = "default"

[[board]]
type = "lamp"
id   = "lamp0"
addr = 8010      # radix 16 -- no 0x needed
```

Nobody taught the TOML loader what a lamp is. It asked the board for its properties, found one called `addr`, and set it. A board added next year is configurable, scriptable, drivable by an assistant, and documented **the day it lands**.

## 5. Test it

`tests/test_lamp.cpp` is the model. It does several things, and the middle one is the important one:

```
// A REAL STORE CYCLE on the bus -- not a method call on the board.
m.bus.memWrite(0x8010, 0x55);
CHECK(lamps(lamp) == 0x55, "the store latched the byte, read back through reflection");

// The board is WRITE-ONLY, and the proof is that the LOAD FLOATS.
CHECK(m.bus.memRead(0x8010) == 0xFF, "a load from 8010 is not the lamp's -- the bus floats");
```

...and then it pins down the claim this whole chapter rests on — that the bus tells the two directions apart:

```
BusCycle store; store.type = Cycle::MemWrite; store.addr = 0x8010;
BusCycle load;  load.type  = Cycle::MemRead;  load.addr  = 0x8010;

CHECK(m.bus.respondersTo(store).size() == 1, "a store at 8010: the lamp, and ONLY the lamp");
CHECK(m.bus.respondersTo(load).empty(),    "a load at 8010: nobody -- the read was never claimed");
CHECK(m.bus.drain().empty(),               "...and the bus logs NOTHING. It is not contention.");
```

**Assert the thing your design depends on, not the thing that is easy to assert.** If the bus ever stopped decoding by direction, this chapter would be teaching a falsehood — and that test is what would say so.

**One trap, and it caught this test.** `m.bus.attach(board)` wires a board to the backplane; it does not hand it to the **machine**. Lifecycle events — `reset()`, `power()`, `pump()` — are dispatched by `Machine` over the boards it *owns*. A board that is only on the bus will answer cycles all day and never hear the RESET line, which is why the test calls `power()` and `reset()` on the lamp by hand. A board that comes from a machine file goes in through `Machine::add()` and gets both.

## 6. What the lamp skipped — wiring a board that ships

The lamp is complete, tested, and configurable, and everything above is the whole story for a board that only latches a byte. A board that talks to the **outside world**, or that a user will find in the manual, needs a few connections the lamp never made. None of them is hard; each is easy to *forget*, and the forgetting fails in a way that does not point back here.

### A board that talks to an endpoint wires its resolver in two places

A board with a real line — a serial port, a printer, a socket — does not open that line itself. It hands a *spec* like `socket:2323` or `file:printout.txt` to a resolver and gets back a `ByteStream`. **The endpoint grammar lives in exactly one file, `src/host/endpoint.cpp` ( `resolveEndpoint` ), and a board must not know it** — that separation is what lets `CONNECT`, tab completion and the MCP schema all speak the same vocabulary without any board learning what a socket is.

A board reaches the resolver through a static `setResolver()` (the SIO family is the model). The trap is that this is wired at the **composition root**, and there are **two** of them:

- `src/main.cpp` — the shipping program.
- `tests/main.cpp` — the test harness.

**Wire the resolver in `tests/main.cpp` too, or your board test connects to nothing.** Both files carry the same block of `YourBoard::setResolver(resolveEndpoint)` lines. A board added to only `main.cpp` works in the running program and then every test that `CONNECT`s it gets a null resolver and fails somewhere unhelpful. Copy the line into both.

## Remember the path as written, not as resolved

If your endpoint is a path ( `file:` , a disk image), resolve it through the board's `resolvePath()` so a relative path means the right thing (relative to the machine file when it came from one, relative to the shell when typed — the rule is in the serial-I/O chapter and the config docs). But **store the spec as the user wrote it** for `describe()` and round-trip.

Where you LOOK is `resolvePath()` ; what you REMEMBER is the path as written. Save the resolved absolute path instead and `CONFIG SAVE` writes *that* back, so the next load rebases an already-absolute path and the one after that rebases again. The hard-sector controller states the rule in a comment at its `openMedia` site; follow it.

## Adding a new endpoint scheme touches its help in two spots

If you are not just consuming endpoints but adding one (a new `something:` scheme in `endpoint.cpp` ), it must appear in `endpointHelp()` and get a one-line explanation in the `CONNECT` command help in `src/cli/commands.cpp` . `test_cli.cpp` asserts that every name `endpointHelp()` offers is a word `CONNECT` explains — a bare `null` or `scripted` tells a user nothing, and the explanation is a hand-copy that rotted once (it promised `socket:` "was coming" long after it shipped). The test is what keeps the two honest.

## Regenerate the reference, ship a machine, and mind the two unguarded docs

Three loose ends after the board itself compiles:

- **The generated board reference.** After editing `registry.cpp` , `commands.cpp` , or a machine file, regenerate `docs/manual/ref/*.md` — `cmake --build build --target docs-reference` — and commit the result. A ctest byte-diffs the committed files and goes red until you do. **Edit the emitter or the source, never the `.md` .**
- **A sample machine is a TOML file**, not C++: drop it in `machines/` with `base = "default"` under `[machine]` and a name. `cmake/embed_machines.cmake` embeds it on the next build.
- **Two documents no test guards, so they drift silently.** The prose board chapter ( `docs/manual/boards.md` ) and the changelog's `Unreleased` section are *not* test-enforced — the manual guard only checks self-containment and `ORDER` membership, nothing checks that the prose lists every registered board. The authoritative list is `registry.cpp` 's `boardTypes()` ; the generated `ref/boards.md` is guarded, but the hand-written chapter is not, and it once drifted nine boards behind before anyone noticed.

When you add a board, update the prose chapter and add an **Unreleased** changelog line by hand. No build will remind you. The board also ships with its own `docs/boards/*.md` (from `docs/boards/_TEMPLATE.md`) and a row in `docs/sources.md` for anything it embeds.

## What to do next

Our board answers a cycle. A more interesting one **asks for something**:

- **Interrupts.** Override `assertsInt()` to report whether your board is pulling the IRQ line, and **call `intChanged()` from every place your pending flag could move.** A needless call costs a virtual call. A missing one hangs the guest forever. It shows up in `SHOW BUS IRQ` for free.
- **Time.** A board with a deadline uses the `Clock` it was handed. A UART absolutely needs one: `transmit-buffer-empty` is a deadline, not a flag.
- **The outside world.** Anything that talks to a socket, a file or a keyboard does it in `pump()` — **never inside a bus cycle.** That seam is what keeps `read()` and `write()` pure computation over state, and it is what a deterministic replay would be built on. The **MP-S serial board** (`src/boards/swtpc-mps.{h,cpp}`) is a good small example that does this for real: a 6850 ACIA whose `tty` unit is a `ByteStream`, so `CONNECT mps0:tty file:out.txt` captures the console and the board never learns what a file is. It is a good next read after this lamp.
- **Sub-units.** A board with a *list* of things — regions on a memory board, drives on a controller — declares `subUnitTables()` and gets `[[board.region]] / [[board.drive]]` in TOML for free.

And whatever you build: **it ships with its `.md`.** A board and its documentation are one deliverable, and the **Limitations** and **Quirks** sections are the load-bearing ones — they are what forces the honest question "*what did I not actually implement?*", which is exactly the question a simulator author most wants to avoid. `docs/boards/_TEMPLATE.md` is the form.

# Serial I/O

Every board that moves characters — the SWTPC MP-S console, the Altair 680b onboard I/O and Universal I/O, the 680b KCACR cassette — is built on the same three layers, and the whole point of them is that a board only ever touches the top one.

**A board knows it has a serial line. It does not know, and must never learn, what is on the other end of it.**

That one sentence is why `CONNECT mps0:tty socket:2323` needs no code in the board, why a board written next year is cross-platform, scriptable and replayable for free, and why there is a whole chapter here instead of a paragraph in each board's source.

The three layers, top to bottom:

| Layer               | Who                                                                  | Owns                                                                                                              |
|---------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| The <b>card</b>     | a <code>Board</code> ( <code>src/core/board.h</code> )               | port decode, register bits, polarity, interrupt straps — everything the <i>manual</i> describes                   |
| The <b>stream</b>   | a <code>ByteStream</code> ( <code>src/host/stream.h</code> )         | non-blocking byte movement, the modem pins, the line rate                                                         |
| The <b>endpoint</b> | <code>resolveEndpoint</code><br>( <code>src/host/endpoint.h</code> ) | the grammar: <code>console</code> , <code>socket:2323</code> , <code>serial:</code> , <code>in:/out:</code> , ... |

A card reaches down to the stream. It **never** reaches down to the endpoint — it does not know the grammar, and it must not learn it.

## The ByteStream contract, and the one discipline

`src/host/stream.h` is the seam. Read it; it is short and every comment in it is load-bearing. The interface a card actually uses:

```
virtual size_t read(uint8_t* buf, size_t n) = 0;    // NON-BLOCKING
virtual size_t write(const uint8_t* buf, size_t n) = 0;
virtual bool readable() const = 0;    // -> drives RDRF / RxRDY
virtual bool writable() const = 0;    // -> drives TDRE / TxRDY
virtual std::string describe() const = 0;    // round-trips the operator's spec
```

The discipline is one line and the whole subsystem stands on it:

**THE BOARD NEVER BLOCKS.** It asks `readable()` / `writable()` and moves on.

A UART that blocked would stop emulated time, and stopping emulated time to wait for a human is precisely how an emulator comes to drop characters. So `read()` returning `0` is **not an error and not an end of file** — it is a quiet line, a poll that found nothing, exactly what an `IN` from a real UART's data port gives you when nobody has typed. A stream never signals EOF up into a board; a board polls forever, because that is what the silicon does.

Two more methods complete the contract, and both exist so a board stays pure:

- `pump()` is called once per time slice **by the run loop, never by the board**. It is the one place a stream may talk to the host: accept a socket, drain a keyboard, poll a file. A card's `pump()` is a one-liner that forwards to `stream_ -> pump()`. Everything a board does in `read()` / `write()` is pure computation over its own state; anything that touches the outside world happens here, at a known point in emulated time. That seam is what would let a recorded session replay identically.
- `drainLog()` is what the far end wants said out loud — a print job that could not be spooled, a serial port that cannot do the baud it was asked for. The board drains it off its units' streams (the default `Board::drainLog()` does exactly this) and the monitor prints it after every command and run.

## The unconnected line is not a special case

```
class NullStream : public ByteStream {
    bool readable() const override { return false; }
    bool writable() const override { return true; }    // TDRE set forever
    size_t write(const uint8_t*, size_t n) override { return n; } // consumed, gone
};
```

An unconnected unit is bound to a `NullStream` — TDRE set, RDRF clear, forever, and a write goes nowhere without complaint. That is exactly how an unconnected 6850 sits on a real card. Because of it there is **no null pointer anywhere in a board's stream path, and therefore no `if (nothing is plugged in)` branch in any board**. Bind the stream to a `NullStream` in the constructor and never let it be null again.

Two more streams are worth knowing because they are how you test:

- `LoopbackStream` jumps TX to RX and loops the modem pins back too (RTS→CTS, DTR→DCD/DSR). It is the one endpoint that tests modem control with no hardware — the plug in the drawer.
- `ScriptedStream` keeps the two directions **separate**: a test types into `feed()` and reads what the guest printed out of `out()`. This is what makes a guest program a unit test — a monitor's banner and its answer to a command are just bytes it wrote to a serial port, and a test that can see them

asserts on them with no terminal, no thread, and no timing in the picture. It also counts empty polls (`hungry()`), which is how the MCP run loop knows a prompt is spinning on input versus a loader that is merely quiet while it works.

## The endpoint grammar lives in exactly one place

```
std::unique_ptr<ByteStream> resolveEndpoint(const std::string& spec, std::string& err);
```

`src/host/endpoint.cpp` is the **only** file in the program that knows the endpoint grammar — `console`, `socket:PORT`, `socket:HOST:PORT`, `serial:DEVICE`, `null`, `loopback`, `in:PATH`, `out:PATH`, `file:PATH`, `printer:QUEUE`. `CONNECT` and `MOUNT` are generic monitor commands, not per-board ones (DESIGN.md §7.7): the monitor opens the endpoint, the board decides what the bytes mean. That division is why a serial card gets every backend the day it lands without one line changing in the monitor.

It never guesses. `CONNECT` `sio:a consle` is an error carrying the list of what it could have meant, not a silent `NullStream` that leaves you wondering why the terminal is dead.

Two helpers exist so a board can *use* the grammar without *reimplementing* it:

- **rebaseEndpointPaths** / **rebasingResolver** rewrite only the PATH-bearing specs (`in:` / `out:`) so a machine-file relative path is resolved against the machine file's directory, not the shell's `cwd`. The grammar of *which* specs carry a path stays in `endpoint.cpp`; the board only supplies its config dir. Remember the path **as the operator wrote it** for `describe()`, and rebase only the copy you hand the resolver — otherwise a relative path double-rebases on `CONFIG SAVE` + reload.
- **parsePort** / **parseHostPort** let a board validate a `HOST:PORT` string using the same split the resolver uses.

## The capture tap is a decorator ByteStream

`ENDPOINT|FILE` taps a line to a hex log — a poor man's protocol analyzer. It is worth studying because it shows how far a `ByteStream` decorator gets you with **zero** board or monitor changes.

`TeeStream` (`src/host/tee_stream.h`) wraps an inner stream, copies every byte past in both directions to a log file, and forwards everything else — `readable` / `writable`, the modem pins, the line rate, `pump`, `pacesItself` — unchanged. It is the same shape as `FilterStream` (`src/host/filter.h`), with one difference that matters:

A **filter mutates bytes**, so there is exactly one of them and it lives on the console (a `strip7out` on a binary transfer corrupts it silently). A **tee never touches a byte** — it observes and forwards — so it is 8-bit clean by construction, and the "one filter, on the console" rule does not apply. You may tap *any* line: a socket, a real serial port, a tape.

The grammar is a single character. `resolveEndpoint` checks for a `|` *before* the prefix dispatch (so `socket:23|cap.hex` is not mistaken for a socket spec), splits on the first one, recurses on the left for the wrapped stream, and parses the right as `FILE[?opts]`. Because the inner is resolved by the same function, the tap composes with everything — `in:tape.tap?cps=300|trace.log` taps a paced paper-tape reader. `describe()` returns `inner->describe() + "|" + fileSpec`, so the whole tap round-trips through `SHOW` and `CONFIG SAVE`, and the `|` re-triggers the branch on reload. Two consequences fall out of the grammar living in one place: `rebaseEndpointPaths` had to learn to split on `|` and rebase **both** sides (a machine-file relative log path is config-relative like any other), and the timestamps come from an **injectable host wall clock** (the printer/tape pattern), never the emulated `Clock` — a trace whose timestamps freeze at a monitor prompt is a trace of nothing.

## Installing the resolver — in both mains

The grammar travels to a board as a function it is handed, never as knowledge it holds:

```
static void setResolver(EndpointResolver r); // on the board (or the Sio2Port section)
```

Install it **once in each composition root**: `src/main.cpp` (the CLI) *and* `tests/main.cpp` (the test binary). Miss the test main and every unit test that connects an endpoint gets a board whose resolver is null. A card that embeds a `Sio2Port` inherits the section's one resolver and needs no `setResolver` of its own.

## The modem pins go both ways, and the chip owns polarity

```
struct LineStatus { bool carrier=true, cts=true, dsr=true; bool ring=false; }; // far end -> card
struct LineControl { bool rts=false, dtr=false, brk=false; }; // card -> far end
```

One rule governs every pin on every stream:

**true** is always "asserted".

The pin-level inversions are real — the 6850's carrier and CTS pins are `/DCD` and `/CTS`, active low — but that is a fact about *that chip's pins*, and it stays inside the chip that has them. A stream that reported

"carrier = true, meaning the pin is high, meaning there **isn't** one" would be exporting one chip's polarity to every other board in the machine, and every board sharing that chip would be wrong for free. The wire carries a level in true sense; the honoring chip decides what it means — the same division the shared **IRQ** line draws, where the wire carries the level and the receiving device decides what to do about it.

A level has **no memory**. The stream says "carrier is down" for as long as carrier is down; the *chip* does the latching (a 6850 holds its DCD flag after the pin returns and clears it only on a status-then-data read). Put the latch in the stream and every stream re-implements it slightly differently.

A console or a file has none of these pins in any real sense, so it **asserts them all** — which is exactly what strapping DCD and CTS to ground on the connector does, and what period installers did constantly. That, again, is why there is no "what if nothing is plugged in" branch anywhere.

## The line rate is the card's, and there is only one

```
struct LineParams { long long baud=9600; int dataBits=8, stopBits=1; LineParity parity=None; };  
virtual bool setParams(const LineParams&, std::string& err);
```

There is exactly one line rate in a serial card and it is the UART's clock — a jumper. When a unit is connected to a **real host serial port**, the *card programs the port*: `SET mps0:tty BAUD=300` restraps the card and the host UART follows. Only a `serial:` endpoint honors `setParams`, and it is the only stream that may **refuse** — an FTDI cable that cannot do 76800 baud is a fact about the world, and the card says so out loud through `drainLog()` rather than run at the wrong speed in silence. A `false` return does not mean the connection failed: the card stays strapped to what it is strapped to and goes on pacing the guest at that rate, because that pacing is the half the guest can actually measure.

A second, independent baud rate on the endpoint was considered and struck. It could only ever configure a *mismatch*, and a 6850 strapped for 300 driving a terminal set to 9600 does not give you a fast link on real hardware — it gives you garbage. Every stream but a real serial port ignores `setParams` entirely: a socket has no baud rate, and pretending it did would pace an emulation against a fiction.

One related flag closes the loop. `pacesItself()` returns true for a stream that carries its own cadence — a cassette releases bytes at a wall-clock rate the CPU's speed cannot drag. When it is true the UART must **not** also impose its emulated line-rate gate, or the two clocks fight (see `Uart1602::poll` and `host/tape.h`). Every other line has no cadence but the one the UART's baud gives it.

## A chip is not a card, and a serial section is not one either

`theory.md` states the chip/board seam: a chip is modeled from its **data sheet**, a board from its **manual**, and the inverting buffers and interrupt-enable flip-flops that live between the chip's pins and the bus are the *board's*. Serial adds one more object in the middle.

A concrete UART is a part in `src/chips/`: `Mc6850` (the MP-S console and the 680b onboard/Universal I/O), `Uart1602` (the COM2502 — the 680b KCACR cassette). A chip knows nothing about the bus; it has a clock, some pins, and a `ByteStream`.

But the *glue* that turns `IN 10h` into "read channel A's status register" — decode, the even→status/odd→data dispatch, the single card-owned Clock deadline, interrupt aggregation, connect/units/properties, SNAPSHOT — is needed by **every** card that carries a 6850, and copying it onto each is how a bug gets fixed on one card and stays wrong on the others. So it lives once, in `Sio2Port` (`src/chips/sio2port.h`), a reusable serial *section*:

```
struct ChannelDef { std::string name; uint8_t offset; }; // the MP-S has {"tty",0}
Sio2Port(std::vector<ChannelDef> channels, std::function<void()> onIntChanged);
```

`Sio2Port` is **not a Board** — no `type()`, no `id`, no bus. The owning board holds one as a member and forwards the bus and lifecycle calls to it (see `src/boards/swtpc-mps.cpp`, one channel). It cannot reach the board's protected `clock_` or `intChanged()`, so it is handed both at construction. Parameterize it by the channels the board has and the whole serial mechanism comes with it.

## TDRE is a deadline, not a flag

A UART does not know a character has finished going out because the guest asked — it knows because *time passed*. So a serial section sets a `Clock` deadline for the moment the byte clears the shift register, and `writable()` /TDRE reflects that deadline, not an instantaneous flag. `pump()` takes an arriving byte off the line; if the line has not yet had time to deliver it, the deadline covers the gap. This is the single most common thing a naive UART gets wrong — read §4.4.1 of `DESIGN.md` before you put any work inside `assertsInt()` (`theory.md` explains why the poll must never be doing the card's clocking).

## Interrupts

The full interrupt model — `assertsInt()` as a pure level, the wire-OR the bus keeps, and `intChanged()` after any pin move — is in `theory.md` and is not serial-specific; a parallel card wires interrupts the same way. For a serial board, the only things to remember:

- Override `assertsInt()` to report the *settled* IRQ pin from the chip's state — a character waiting with the receive interrupt enabled asserts, and keeps asserting until the guest reads the character. It is a level; there is no queue to lose.
- Call `intChanged()` wherever the pending flag can move: a register written, a byte taken off the line, a deadline coming due. A missing one hangs the guest forever waiting for an interrupt that already happened — and it presents as "*the emulator locks up sometimes*".

An embedded `Sio2Port` aggregates its channels' pins and drives the callback the card bound to its own `intChanged()`, so a card that forwards to a section gets this right by construction.

## The chip-backed serial board

A serial board emulates a specific UART. It embeds a `Sio2Port` (or a bare chip), forwards bus/lifecycle/connect to it, and adds only the board-specific glue: the base-port jumper, the inverting buffers, the interrupt straps. The MP-S and the 680b I/O boards are this. The manual tells you the polarity and the port map; the data sheet tells you the chip — and the project rule stands: never give the hardware a behavior it never had to make software happy.

## How to add a serial board

The general playbook is `adding-a-board.md`; the serial-specific steps on top of it:

1. **Own a stream, never null.** Hold a `std::unique_ptr<ByteStream>`, bind it to a `NullStream` in the constructor. Or embed a `Sio2Port` and let it hold the streams.
2. **`readable()` → RDRF, `writable()` → TDRE.** Synthesize your status byte from those; apply your card's polarity here, in true sense at the pins.
3. **Expose one serial unit per channel** (`units()` → `UnitKind::Serial`), wire `connect/disconnect/unitStream`, and route `pump()` to the stream.
4. **Install the resolver in both mains** — `src/main.cpp` and `tests/main.cpp` — via your board's (or `Sio2Port`'s) `setResolver`.
5. **If you touch a real serial port**, push `LineParams` through `setParams` on connect and on any baud change, and surface a refusal through `drainLog()`.
6. **Model real silicon:** put the chip in `src/chips/` from its data sheet, keep the inverting buffers and interrupt-enable bits on the board, and reuse `Sio2Port` if it is a 6850.
7. **Test with a `ScriptedStream`** bound through the real `connect()` path: `feed()` bytes at the card, assert on `out()`, and check the status bits land where the straps put them.

Then regenerate the board reference (`cmake --build build --target docs-reference`) so the generated `ref/boards.md` picks up your properties, and add the board to the changelog and the prose board chapter — neither is test-enforced, so both drift silently.

# The built-in terminal

---

`CONNECT mps0:tty terminal` gives a serial line a windowed VT100 (or ADM-3A, VT52, H19) that the simulator draws itself — no telnet client, no external emulator. This chapter is how that is built, and how to add an emulation.

It is a **serial endpoint**, so read `Serial I/O` first: the terminal is one more `ByteStream` behind `resolveEndpoint`, and no board knows it exists. What is new is that the stream has a **screen** and a **keyboard**, and both live in the host `Display`.

## Two halves: the screen engine, and the dialect

The terminal engine is `src/host/terminal/`, and it is deliberately split so that the part that differs between terminals is small:

| Piece                         | File                    | Owns                                                                                                                                                                                                                                                |
|-------------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>TerminalScreen</code>   | <code>screen.h</code>   | the attributed character grid and its primitive ops ( <code>putGlyph</code> , <code>lineFeed</code> , <code>eraseToEol</code> , <code>cursor</code> ). Sized ( <code>rows</code> , <code>cols</code> ) at construction — nothing is baked to 80×24. |
| <code>TerminalEmulator</code> | <code>emulator.h</code> | the <b>dialect</b> : a byte→ops state machine ( <code>feed()</code> ), plus the reply FIFO and key encoding. This is the only piece that changes per terminal.                                                                                      |
| <code>TerminalRenderer</code> | <code>renderer.h</code> | paints a <code>TerminalScreen</code> into a host <code>Display</code> , once per frame, using a <code>TerminalFont</code> .                                                                                                                         |
| <code>TerminalFont</code>     | <code>font.h</code>     | the glyph source. The bundled default is the authentic <b>DEC VT220</b> face ( <code>boards/terminal-vt220font.h</code> , a 10×20 cell); a board with its own character generator could supply a CGEN PROM through the same seam.                   |

**This engine began as a built-in video terminal board** (issue #244, Task 1) — a card with its own screen and CGEN PROM that was later split into this reusable screen/dialect engine. That board did not survive the move to the 6800 machines (it was S-100 hardware), but the engine did: today it backs the generic `terminal:` endpoint alone, reached through a serial `ByteStream` with a pluggable dialect. So any serial line — the SWTPC MP-S console on the SS-30 bus, the 680b's second port — can be given a windowed terminal without a board learning what a screen is.

`TerminalFont::glyphRow()` returns a `uint16_t` scan line with **bit 15 the leftmost dot**, so a cell may be wider than eight dots — the VT220 face is 10 wide. An eight-dot font MSB-aligns its byte into the top of the word, keeping bit 15 leftmost for every font. The built-in terminal's phosphor palette (green default, amber) is set on its `TerminalRenderer` by `TerminalStream::setPhosphor()`, driven by the `phosphor=` connect-string option.

## The emulator contract

`TerminalEmulator` (`emulator.h`) is short and worth reading in full. The load-bearing part is that a terminal is **bidirectional**, and everything flowing toward the guest goes through one FIFO:

- `feed(byte, screen)` — apply one display byte; advance any multi-byte escape sequence. May enqueue a **reply** (a report the guest asked for, e.g. a VT100's `ESC[6n` cursor position).
- `keyAscii(byte)` — a host keystroke with an ASCII code. The base passes it straight through; an emulator overrides only to remap.
- `keySpecial(Key)` — a host key with **no** ASCII (`Key:: {Up, Down, Left, Right, Home}`). This is the classic difference: an arrow is `ESC[A` on a VT100, `ESC A` on a VT52, `Ctrl-K` on an ADM-3A. The base sends nothing; an emulator with arrows overrides it.

Reports and keystrokes both land in the **reply FIFO**, drained with `takeReply()`. That is correct: to a UART, a report and a keystroke are both just characters arriving on the line, and the guest cannot — must not — tell them apart. `reset()` abandons any partial sequence and clears the FIFO, as a bus RESET does.

## Registering an emulation

The dialects are a name→factory table, `emulations.{h,cpp}` — the one place that knows which terminals exist, mirroring `serialBuiltins()`. Adding a terminal is:

1. Write the `TerminalEmulator` subclass (e.g. `src/host/terminal/vt52.h`).
2. Add one row to the table in `emulations.cpp` (`{ "vt52", "DEC VT52", &makeVt52 }`).
3. Give it grid-assertion tests in `tests/test_terminal.cpp`.

The endpoint grammar, the `HELP CONNECT` gloss and the error messages all read this table, so nothing else changes. **The first row is the default** when a `terminal:` names no emulation (`vt100`). Names match case-insensitively, like a board id or a property.

The four shipped dialects: `vt100 / ansi` (a working editor subset — CSI moves, ED/EL, SGR, `ESC[6n`, DECSC/DECRC, DECCKM arrow flip); `adm3a` (the dumb terminal — control-code moves, `ESC =` cursor addressing, **no reports**, because the hardware had none); `vt52`; and `h19`, which is a VT52 superset in Heath mode and **delegates to `Vt100Emulator`** in ANSI mode (`ESC <` enters it, `ESC[?2l` DECANM returns).

## The stream and the endpoint

`TerminalStream` (`stream.{h,cpp}`) is the engine wearing a `ByteStream` face:

- `write()` — guest → screen: feed each byte to the emulator.
- `read()` — guest ← the emulator's reply FIFO (reports **and** the keystrokes routed to this line).
- `pump()` — paint one frame into the host `Display`, on the main thread.

The `Display` and `TerminalFont` are **injected once at the composition root** (`setDisplay` / `setFont` in `src/main.cpp`), the same way the boards' `Display` and the endpoint resolver are — a stream built deep inside `resolveEndpoint` cannot be handed them, and they are session-lifetime host resources it only borrows.

The `terminal:` scheme is parsed in `resolveEndpoint` (`src/host/endpoint.cpp`): grammar `terminal[? emulation=vt100&size=80x24]` (`size` also spelled `windowSize`), validated **before** the window check, so a typo is a grammar error rather than a mysterious refusal. Then capability: a terminal needs a window, so `TerminalStream::hasWindow()` gates it — `SdlDisplay` answers `isWindowed()` true, `NullDisplay` false, and a headless build refuses `terminal:` cleanly at `CONNECT` rather than opening a serial line nobody can see. SDL is optional throughout: absent it, the endpoint is simply unavailable, never a compile break.

## The one keyboard, and where it goes

Here is the constraint that makes this bigger than the printing endpoint. Today there is exactly **one** `SdlDisplay g_display`, shared by every video board, with **one** global key sink. The model is "one window, one keyboard." A `terminal:` line renders in that window and must receive its keystrokes — but so does the monitor, and so would a video board if the machine had one.

The routing (issue #244, Task 4):

- `TerminalStream::keyTarget()` names which terminal line owns the keyboard. A `terminal:` claims it on construction — **last one wins** — and its destructor releases it *only if it is still the owner*. That ordering matters: a `CONFIG LOAD` constructs the replacement line (which claims the target) **before** destroying the old one, so a blind clear in the destructor would strand the new terminal (see the `config-load-replaces` rule in `DESIGN.md`).
- The composition root's key sink (`src/main.cpp`) routes regular keys to `keyTarget()->keyAscii()`, falling back to `Console::instance().inject()` when no terminal is present — so the monitor keyboard behaves exactly as before.
- Arrows/Home carry no ASCII, and the `Display` normally collapses them to a single control *byte* before any sink sees them (`emitSpecialKey` in `src/host/display.h`) — which a terminal could never re-encode in its own dialect. So there is a second seam, `Display::SpecialKeySink`, delivering the key **symbolically**; when set it takes precedence over the byte table, and the composition root routes it to `keyTarget()->keySpecial()`, falling back to injecting the table byte into the `Console`.

**Deferred:** independent windows with per-window keyboard routing. That is the real fix for two concurrent terminals — and for the keyboard contention between the monitor and a terminal in one machine — and it is the last unbuilt piece of the plan. Until a two-terminal machine needs it, one host window drives whichever line claimed it last.

## The [terminal] transform chain

`src/host/filter.h` argues at length that the console's fold-the-bytes transforms (`strip7out`, `upper`, `bsdel`, ...) must **not** live on a serial line: a line also carries XMODEM, and a `strip7out` there corrupts binary silently. But that file names one exception in passing — *only the console transforms, because only the console has a human on the end of it, and every one of these transforms is a fact about a **terminal***. The built-in terminal is exactly that: a terminal with a human on the end. So it carries the same chain, as the [terminal] config section.

It exists because of a concrete bug (issue #244). An even-parity monitor — the MITS Programming System II — computes parity **into bit 7** of every character it prints. A carriage return `0x0D` goes out as `0x8D`; a line feed `0x0A` is already even and stays `0x0A`. A `terminal:` line is raw 8-bit, so `Vt100Emulator::feed` sees `0x8D`, which is `>= 0x20`, and prints it as a glyph instead of homing the cursor — while the `0x0A` still line-feeds. The symptom is "Enter feeds a line but never returns." `strip7out` masks bit 7 and both bugs vanish at once.

Design points, all in `src/host/terminal/stream.{h,cpp}`:

- **Modeled on [display], not on a board.** `TerminalStream::Settings` is a `static`, and `TerminalStream::properties()` publishes it through the one `Property` layer — so SET, SHOW, TOML load and tab-completion all pick it up with no new plumbing. Config binding is at `src/config/toml.cpp` (a [terminal] branch beside [console] / [display]), and the monitor SET/SHOW/ `showTerminal` sites mirror `display`. Like [console], it does **not** round-trip through `CONFIG SAVE`, and it is accepted on a headless build (the setting is about what the machine wants).
- **A live-read static, global.** One section for the machine, like [console]; `SET terminal strip7out=on` reaches the running window because `write()` / `keyAscii()` read the static each time. `BsMap` is reused from `host/filter.h` rather than re-declared.
- **Outbound folds in `write()`, inbound folds in `keyAscii()` — NOT `read()`.** This is the one non-obvious call. `read()` drains the emulator's reply FIFO, and that FIFO carries the cursor reports the guest asked for (`ESC[6n` → `ESC[r;cR`) as well as encoded keystrokes. Folding in `read()` would mangle a report — an H19 `ESC n` reply can contain a lowercase coordinate byte, which `upper` would corrupt. Keystrokes enter through `keyAscii()`, so that is where `upper`, `strip7in` and `bsdel` belong; reports never pass through it. `test_terminal.cpp` has a guard for exactly this.
- `cr = cr | crlf` is the terminal's one line-ending knob (the console spells the same idea `crlf=on/off`). Default `cr` passes the guest's CR through untouched; `crlf` appends an LF after each CR, applied *after* `strip7out` so it triggers on the real `0x0D`.

## The run banner and a non-stdio console

A related fix rode along. `Monitor::runFrom` announced `(no console connected)` whenever the console was not the **stdio** one — so a machine consoled on a `terminal:` window or a `socket:` was mislabeled as having no console at all. The banner loop already computed `anyRemoteLine` (a live line that is not the host terminal); it now also captures that line's scheme and prints `(console on terminal) / (console on socket)`. The honest `(no console connected)` is kept for the genuinely bare backplane — the ROM-talking-to-a-disk case — where no serial line is live.

## Testing

`tests/test_terminal.cpp` drives the engine directly through a `NullDisplay`: feed a dialect's escape sequences, assert the grid ( `screen()` ), and assert `read()` returns the expected reports and key encodings. It needs no window, because the display seam is only exercised in `main.cpp`. The `[terminal]` transforms have their own section there (including the report-not-folded guard). Run it with `./build/swtpc_tests terminal`; the CONNECT grammar, the `[terminal]` SET/SHOW surface and the run-banner label are in `./build/swtpc_tests cli`.

# Soft-sector floppy controllers

Every soft-sector floppy board — the SWTPC DC-4, and whatever WD179x-based board turns up next — is built on the same three layers, and the whole point of them is that a board only ever touches the top one and reuses the two below unchanged.

**A board decodes its ports and straps its chip. It does not know what a sector looks like, where a track's bytes live in the file, or how a format is parsed. Those belong to the chip and the drive, and they are shared.**

That one rule is why adding the DC-4 was mostly a slot decode and a drive-select latch, and why this chapter exists once instead of a copy in each board's source. (For the *serial* seam this mirrors, read `serial-io.md` first — the shape is identical.)

The three layers, top to bottom:

| Layer     | Who                                                                                                                                   | Owns                                                                                                                           |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| The board | a <code>Board</code> ( <code>src/core/board.h</code> , e.g. <code>boards/swtpc-dc4.h</code> )                                         | port/slot decode, register bits, the drive-select/side latch, the <b>density strap</b> ( <code>DDEN</code> ), interrupt straps |
| The chip  | <code>Wd17xx</code> ( <code>src/chips/wd17xx.h</code> )                                                                               | the register file and the Type I/II/III command FSM; <code>dataRateBits</code> is the <code>DDEN</code> pin                    |
| The drive | <code>DiskImageDrive</code> ( <code>src/boards/floppy-drive.h</code> ) over a <code>DiskImage</code> ( <code>src/host/disk.h</code> ) | CHS ↔ file offsets, synthesized ID fields, the <b>format parse</b>                                                             |

A card reaches down to the chip ( `Wd17xx::attach` ), the straps). It never reaches into the `DiskImage`; the drive is the only thing that touches it.

## The chip: `Wd17xx` and the parts

`src/chips/wd17xx.h` is the FD177x family. Read its header — every comment is load-bearing, and the chip/drive split is spelled out there at length. The essentials for a board author:

- **The class is the part, the file is the family.** `Wd1771` is single density (FM); `Wd1791` is single *and* double density (FM/MFM), with a side-select pin and a one-bit record type. The base `Wd17xx` is the whole register file and command FSM; a part supplies only the four things that genuinely differ (step-rate table, Read-Address side byte, record-type bits, and which data-address-mark a Write writes). Build the part the card has; do not `#ifdef`.
- **No drive select, no side select, no motor.** The chip talks to ONE `FloppyDrive`; the card's select latch points it with `attach()`. Side is a card latch too ( `setSide` ).

- **dataRateBits** is the **DDEN** pin. 250 kbit/s is 8" single density; a double-density card writes 500 kbit/s when it decodes its density bit. The chip uses it for byte timing **and hands it to the drive's Write Track calls**, which derive the revolution byte budget and the recorded per-track density from it. The board sets it from its control-port density bit. This is the **single source of truth for density** — do not duplicate it onto the drive.
- **Wait-synced vs DRQ-polling.** A card whose data port stalls the CPU on a wait-state generator (PRDY) sets `setWaitSynced(true)`; then every command completes on the register access that would have stalled, one byte per access, and Lost Data is correctly unreachable. The DC-4 in `full` mode is wait-synced; in `real` mode it leaves it off and gets byte timing.

## The drive: **DiskImageDrive** over **DiskImage**

`src/boards/floppy-drive.h` is the generic adapter between the chip's pins and a flat logical `DiskImage`. A raw `.DSK` holds **sector payloads only** — no gaps, no address marks, no CRCs (`src/host/disk.h`). So:

- **ID fields are synthesized** from the mounted image's declared per-track `TrackFormat` (`sectorIdAt`): the track is the physical head position, the sector counts from the format's `startSector` (1 on a soft-sector card), and the CRCs always check (an image carries no rot).
- **DiskImage is CHS with per-track geometry.** Each `(track, head)` slot carries a `TrackFormat{density, sectors, sectorSize, startSector}`; offsets are a **running sum** over the slots (`rebuild()`), so a disk whose tracks differ in size — the mixed-density disk — is expressible where one global geometry could not say it.
- **The head position lives in the drive, not the chip's Track Register.** They may disagree; that is what a verify catches (Seek Error).

## Geometry in real time — the FORMAT path

This is the reusable core. A soft-sector disk's geometry is **not** fixed at mount: sector size, count and density can vary track to track, and none of it is in the `.DSK`. So:

**Write Track is the only command that establishes or mutates a track's geometry.** Reads and writes never do; they *validate* against what a track records.

The chip side is already done (`Wd17xx`, no per-board work): on `Write Track` the chip asks the drive for `trackImageBytes(dataRateBits)`, and if it is positive it enters the write phase, accumulates every guest byte into an internal buffer, and hands the whole revolution to `drive->writeTrackImage(buf, dataRateBits)` at the end. Both calls carry the chip's own configured data rate — the chip is the single source of truth for density, and the drive keeps no copy: it derives the revolution byte budget and the

recorded density from the rate passed in. When `trackImageBytes(rate)` is 0 the chip sets **WRITE FAULT (S5)** instead — the honest answer for an empty drive or a controller that does not format.

`DiskImageDrive::writeTrackImage` (in `floppy-drive.cpp`) is the format FSM, run over the whole collected buffer:

```
gap ... 0xFC(index) ... gap ... 0xFE track side sector N 0xF7 ... gap ... 0xFB <data...> 0xF7 ... gap ... (repeat)
      ^ID address mark ^length code                ^data address mark ^CRC-generate
```

Per sector: `sectorSize = 128 << N`, capture the first sector number as `startSector`, and **accumulate the data field until 0xF7** — the CRC-generate byte, which can never appear as literal track data, so `accumulate-until-0xF7` is unambiguous (the `0xE5` fill and the `0xDD` density signature are ordinary data, not special). Then:

1. Derive `TrackFormat{density = (rate ≥ 500 kbit/s ? DD : SD), sectors, sectorSize, startSector}` and call `img->setTrackFormat(head, side, tf)`. That marks the slot valid and re-runs `rebuild()`, so the following tracks' offsets — and the growth cap — follow.
2. Write each sector's payload by a **sequential 1..N counter** from `startSector`, ignoring the header's possibly-skewed sector number, so the fill stays contiguous and the file grows in order. **Write exactly the bytes the guest streamed — never fabricate or pad the fill.** A data field shorter than the recorded `sectorSize` is a malformed track and `writeSector` rejects it → **WRITE FAULT**, which is honest.

Return `false` (→ **WRITE FAULT**) only if nothing parsed or a write could not land.

## The ascending-track-order invariant

Correctness rests on one fact: **FORMAT writes tracks 0→N in order**. So when a track is (re)formatted to a *larger* geometry, `rebuild()` moving the following tracks' offsets clobbers nothing valid — they have not been written yet, or are about to be overwritten. A real format program formats tracks ascending. A fully-correct out-of-order / cross-density reformat of an *already populated* disk would have to shift the tail by the size delta first; that is **deferred** (see below). (FLEX's `NEWDISK` formats ascending, as period format programs do.)

## Growth: `setExtendsOnWrite` and the dynamic cap

`DiskImage::setExtendsOnWrite(true)` lets the backing file grow as sectors are written, capped at `geometryBytes_`. For a soft-sector card that cap is **dynamic** — it rises as each track is formatted (`setTrackFormat` → `rebuild`). A recognized full disk never grows (its writes stay in bounds); a blank one grows track by track as it formats. The board turns it on at mount.

## The density model

Density is one value with three faces, and they must agree:

| Face                           | Where                                                                  |
|--------------------------------|------------------------------------------------------------------------|
| The <code>DDEN</code> pin      | <code>Wd17xx::dataRateBits</code> (250 kbit/s SD, 500 kbit/s DD)       |
| The board control bit          | the board's control-port/latch density bit                             |
| The recorded per-track density | <code>TrackFormat.density</code> , written by <code>Write Track</code> |

The board sets `dataRateBits` from its control-port bit; the chip hands that same rate to the drive's `Write Track` calls; the drive **records** the density it implies into `TrackFormat`. So a double-density board formats a mixed disk from the guest's per-track density bit, and it also *reads* plain single-density media. Cross-reference the WD `WD177X-00` datasheet's `DDEN` description.

Reads and writes **validate geometry** — the addressed track's recorded sector layout via `locate()` — and return **Record Not Found (S4)** on an unformatted / out-of-range track, never WRITE FAULT. The **format path records density; it is never density-gated**.

**Read-side density gate: still deferred.** Reads validate the *geometry* a track records, not its density: a track formatted DD but read with the controller strapped SD still reads back its bytes. Real software never does this (DIFORMAT sets the density bit for the whole DD pass, the boot path reads track 0 SD), so the gate buys nothing yet. When a workload needs it, compare `dataRateBits` against `TrackFormat.density` in the read path and RNF on a mismatch.

## Mount vs. format — where geometry starts

`describeGeometry` (the board's size probe) establishes the *initial* geometry so a recognized disk is usable immediately with no FORMAT:

- A recognized size → that format (padding-tolerant via `sizeMatches`, for the XMODEM pad).
- **A blank / short image → an *unformatted* disk**, mounted at the card's track count with **empty** per-track geometry (no ranges): READY and steppable, every access RNFs until `Write Track` lays a track down. This is what makes `MOUNT ... CREATE` (a 0-byte file) formattable. Empty-pending is preferred over fabricating slots, since `Write Track` is the sole source of geometry. The default rule: *anything that is not a recognized size is single density*.
- Oversized / garbage is still an error — a real track is never larger than one revolution.

A dual-density controller's probe is a **superset**, not a single-size gate: the DC-4 recognizes its double-sided FLEX disk (204,800), a single-sided one (102,400), and a blank (unformatted, formattable) — because the DD controller genuinely reads SD media too.

## The `trackImageBytes(rate)` budget — the load-bearing number

Under wait-synced operation there is no index-pulse timeout: `Write Track` completes **exactly** when the collected buffer reaches `trackImageBytes(rate)`. So that value is the per-track raw byte budget, and it is the one number most likely to bite:

- **Too large → the command hangs**, waiting for bytes the guest will never send.
- **Too small → the last sectors truncate**, because the command commits before the guest has streamed them.

It is derived from the chip's data rate **and the drive's rotation speed** — one revolution is `rate / (8 × rev/s)` bytes (`rate/8` bytes per second ÷ the revolutions per second). An 8" drive turns at 360 RPM = 6 rev/s: **5208** at 250 kbit/s (8" SD) and **10416** at 500 kbit/s (8" DD). A 5.25" mini turns at **300 RPM = 5 rev/s**, a longer revolution — `rate / 40`: **6250** (5.25" SD) and **12500** (5.25" DD). The RPM is a physical property of the drive, so the card sets it per mounted drive from the diskette's size (`DiskImageDrive::setRevsPerSecond`, default 6 — an 8"-only board that never sets it is unchanged); the chip stays the single source of the rate.

The budget must be  $\geq$  everything the format program streams before its trailing gap. A format program streams its structured bytes then pads with `0xFF` until the controller signals INTRQ — which is exactly when the buffer hits the budget. Because the rate is the chip's, one mechanism gives a double-density board 5208 for an SD track and 10416 for a DD track, and a 5.25" FLEX disk 6250/12500. Validate this number against the format program's gap tables, not by "it booted."

## The flat-`.DSK` limitation

A raw `.DSK` cannot record varied per-track geometry — it is payload bytes only. So the geometry is **re-derived from the file size on every remount** (`describeGeometry`), which is fine for a uniform SSSD disk and the one standard mixed disk, but a `.DSK` that had been formatted to some *custom* per-track layout would lose it across a remount. An IMD/TD0-style container that carries its own sector map would fix this — and is explicitly never coming (DESIGN.md §7.3): such files are converted to raw beforehand.

## Deferred

- **Shift-tail resize** — on a `Write Track` that changes a *populated* track's total size, `memmove` the following data by the size delta before `setTrackFormat`, so a partial / out-of-order / cross-density reformat stays correct. Not needed for blank format or a whole-disk ascending reformat.
- **Read-side density gate** (above): reads validate geometry, not density. Deferred until a workload needs it.
- **A guest-driven FLEX NEWDISK acceptance test.** Formatting is covered at the **board level** in `tests/test_swtpc_dc4.cpp` — the register discipline, the file growing, and the fill reading back at the right per-track geometry — and `acceptance-flex` boots FLEX 2.0 off a tracked disk end to end. A guest-driven interactive `NEWDISK` run is feasible but slow and stale-buffer-prone over the console, so the guaranteed proof stays the board test.

## Adding another soft-sector controller — the checklist

1. **Build the right WD part** in the board's `buildChip()` — `Wd1771` for a single-density card, `Wd1791` for one that does double density — and `setWaitSynced(true)` if the data port stalls the CPU.
2. **Strap density** from the control-port bit into `chip_>dataRateBits` (leave it at 250 kHz for an SD-only card).
3. **Size-probe with a blank fallback** in `describeGeometry`: recognized sizes → their format; anything smaller → empty per-track geometry (unformatted); oversized → error.
4. **`setExtendsOnWrite(true)`** on the image at mount, and enable formatting on the drive (`setFormatting(true)`) — leave it off (the default) on a card that does not format, and `Write Track` keeps faulting. Neither the byte budget nor the density is passed here: both are the chip's, derived per call from the data rate it hands the drive.
5. **Reuse `DiskImageDrive`'s format path** unchanged — the parse, the sequential fill and the `setTrackFormat / rebuild` are controller-agnostic.
6. **Set the drive's RPM** (`setRevsPerSecond`) at mount if it holds anything other than 8" media — the budget denominator. Skip it for an 8"-only card (the default 6 is correct).

## The worked example

- **SWTPC DC-4** (`src/boards/swtpc-dc4.cpp`) is the shipped controller: a `Wd1791` (FM/MFM) bound to a `DiskImageDrive`, **memory-mapped on the SS-30 bus** rather than decoding I/O ports — a drive-select/side latch at one slot and the WD179x register block at the next. It probes the FLEX geometries (35- and 40-track single-sided, and double-sided) with a blank fallback, sets `setRevsPerSecond(5)` for its 5.25" media (step 6), and turns `setExtendsOnWrite / setFormatting` on at mount so FLEX's `NEWDISK` can format a blank image. It also **coerces a boot read of sector 0 to sector 1**, because FLEX numbers sectors from 1 while the boot loader clears the sector register

before its first read — the one place the board rewrites what the guest asked for, and its comment says why.

- **Double-sided blank-grow rides the ascending-slot-order invariant.** A blank double-sided disk grows contiguously only if the guest's format order matches the image's slot layout, so `setExtendsOnWrite` never has to fill a gap. FLEX lays a double-sided disk **cylinder-major** (side 0 then side 1 of each cylinder before stepping), which agrees with the slot layout, so it is safe. A format program that wrote all of one side before the other under this same layout would leave gaps and need the shift-tail resize (still deferred).

# The PS II monitor loader, inside

The MITS **Programming System II** — Processor Technology's editor, assembler, monitor and debugger for the Altair — boots from cassette in a way that repays a close read, because the mechanism it uses to bring in its monitor is the same mechanism the rest of the package uses to bring in everything else. Booting the monitor is **two stacked loaders**: a tiny hand-toggled bootstrap that installs a block-load interpreter, and that interpreter reading the monitor itself. Once you see the second one you have the whole Software Package II — the editor, the assemblers and the debugger are nothing but data for it.

**The payoff of digging through the bootstrap is the block-load protocol underneath it.** The bootstrap is a one-time trick to get that interpreter into memory when nothing yet can interpret anything. Everything after boot — including every tool you EDT, ASM or DBG from tape — is addressed, checksummed blocks fed to the same resident engine.

The source assets live in `tapes/MitsPS2/`: `LDRPS2.ASM` / `LDRPS2.HEX` (the toggle-in bootstrap), `PS2-MON.TAP` (the monitor tape), `PS2-EDT.BIN` / `PS2-ASM.BIN` / `PS2-AM2.BIN` / `PS2-DBG.BIN` (the tools), and the operator machine files `ps2.toml` and `ps2int.toml` (the second is the interrupts variant — an 88-VI/RTC and sense switch A9). Boot either with `swt pcsim ps2` / `swt pcsim ps2int`; to watch the loader run, drive it over `--mcp` and set a breakpoint (see `docs/manual/mcp.md`).

## Layer 1 — the hand-toggled bootstrap

`LDRPS2.HEX` is the twenty bytes you would toggle in at `0x0000` on real hardware. It reads the 88-ACR one byte at a time and copies the second-stage loader **downward** into `0x0F00–0x0FAD`, then jumps to it. The whole thing is in `tapes/MitsPS2/LDRPS2.ASM`; the core is a five-instruction loop:

```
loop    lxi    h,0FAEh      ; H=0F (page), L=AE (=174 = leader byte = byte count)
        lxi    sp,stack    ; init SP so a RET jumps back to loop
        in     06h         ; ACR status
        rrc                ; new byte ready?
        rc                ; no -> RET -> loop
        in     07h         ; the byte
        cmp    l           ; is it the leader byte?
        rz                ; yes -> RET -> loop (skip leader)
        dcr    l           ; not leader: step the address down
        mov    m,a         ; store it (reverse order)
        rnz                ; more to go? -> RET -> loop
        pchl                ; L hit 0 -> jump to 0F00, the code we just loaded
```

The trick worth naming is that **register L does triple duty**: it is the expected leader byte, the remaining byte count, and the low byte of the descending store address, all at once. That only works because the payload is exactly **174 bytes** and the tape's leader byte is `0xAE` — the same number. `H:L` starts at `0x0FAE`; each stored byte decrements `L`; when `L` reaches zero the last byte has landed at `0xF00` and `PCHL` jumps there. One register, because the tape was authored so the three quantities coincide.

This is the **4K BASIC v3.2** bootstrap (`lxi h,0FAEh`), *not* the 8K one. The distinction is invisible from the tape: the leader byte `0xAE` is the low byte of the load address, so it is identical for the 8K loader (which would place the code a page higher) — nothing on the tape tells the two apart, and neither can you until you disassemble what landed. Load this second stage 4K too high and every one of its own jumps lands in the empty page below it and the machine wanders off silently. The `LDRPS2.ASM` header spells this ambiguity out at length; the `ps2.toml` / `ps2int.toml` comments repeat the warning where an operator meets it.

## Layer 2 — the block-load protocol

The 174 bytes that land at `0xF00` are the real loader, and it is general-purpose. It:

1. **Reads the sense switches** (`IN 0FFH`) to learn which serial port pair the console is on.
2. **Self-modifies its own `IN` instructions** — it patches the port operands of its own read instructions to the UART pair the switches selected, so one image serves whichever card is present. (This is why a linear disassembly of the loader as it sits on tape is only *half* the story: some of its bytes are rewritten before they run. See the next section for the sharpest instance.)
3. **Runs a command loop** over two commands read from the console/tape stream:
  - `'<'` — load a block: a length, a load address, that many data bytes, and a checksum. The block is range-checked, stored, **read back and verified**, and checksummed before the loader accepts it.
  - `'x'` — read an address and **execute** (jump to it).

`PS2-MON.TAP` after the bootstrap payload is exactly this: eleven `'<'` blocks, the low RST-vector block loaded **last**, and a final `x 0040` that warm-starts into the freshly assembled monitor. The monitor is not a single image with an entry point of its own — it is a handful of checksummed blocks and a jump, delivered by the interpreter the bootstrap just installed.

## The `0xF27` byte — filler *and* code

While reading the second stage under `DISASM`, one byte reads as nonsense: `0xF27`. It looks like filler that only exists to make the loader assemble, and removing it "to clean up the disassembly" is exactly the wrong move — because **the byte is both filler and code, decoded two different ways depending on how control reaches it**.

The sense-switch setup ahead of it is a chain of conditional jumps, one per valid switch setting. Two paths run *through* this region:

| Reached at | Bytes    | Decodes as | When                                             |
|------------|----------|------------|--------------------------------------------------|
| 0F27       | DA 06 06 | JC 0606    | all the preceding conditional jumps fell through |
| 0F28       | 06 06    | MVI B,06   | a preceding conditional jump landed on 0F28      |

The single byte `DA` at `0F27` is either the opcode of a `JC` (when execution flows straight into it) or skipped entirely (when a jump lands one byte past it, on the `06 06` that then reads as `MVI B,06`). No linear disassembler can render this honestly: a disassembly assigns each byte to exactly one instruction, and this byte belongs to two. That is the real answer to "why does the loader disassemble into garbage here" — the garbage is inherent to an overlapping instruction, not a defect in the tool and not a region you could mark as "data" without lying the other way. Following it needs a breakpoint at the main-loop entry and a walk through each valid sense-switch path ( `docs/manual/mcp.md` ), not a static listing.

One theory that did *not* survive the digging: that the filler exists to shift the byte stream past a leader-match collision, or that the monitor reuses this second-stage loader to bring in its extensions. The reuse theory is disproved directly — loading the editor overwrites `0x0F00–0x0FAD`, where the second-stage loader sits, so it cannot still be resident to be reused. The extensions come in by a different door.

## The tools are pure block-load data

That different door is the block-load interpreter, still resident from the monitor boot. The editor tape is the clean example. `PS2-EDT.BIN` (1920 bytes) has **no leader trick and no second-stage loader** — just an `EDT` name fragment, eight `'<'` blocks landing at `0x0A40–0x1150`, and a closing `x 0A40`. It is nothing but addressed, checksummed data for the engine the monitor already installed; live, an `EDT` load reads 1910 of the 1920 bytes (the last ten are trailing padding) and a dump at `0x0A40` matches the tape byte for byte.

This is the I/O-Table design the PS II manual describes. `EDT` loads a program of that name from wherever the symbolic name `ABS` points — `ABS` defaults to `TY` (the console), which is why `OPN ABS,AC` must first repoint `ABS` at the cassette driver before the load will come off tape. The same shape covers the rest of the package: `PS2-ASM.BIN`, `PS2-AM2.BIN` and `PS2-DBG.BIN` are all block-load data over the resident interpreter. Understand the monitor boot and you have understood how every one of them arrives.

# Adding a built-in ROM ( `.S19` → embedded image)

A built-in ROM is a directory under `roms/<NAME>/` that the build compiles into the binary, reachable from a machine file as `mount = "builtin:<name>"`. Every 6800 monitor this simulator ships — `SWTBUG`, `MON680`, the `KCACR` cassette loader — is one, and **none of them is assembled here**. They arrive as a **Motorola S-record** image, verbatim from the period source ([deramp.com](http://deramp.com)), and are embedded byte-for-byte. The developer's job is to place the image, reconcile it against the listing, and wire it in — not to run an assembler.

## What goes in the directory

`roms/SWTBUG/` is the pattern:

| File                    |                                                                                                                                  |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <code>SWTBUG.S19</code> | <b>The image.</b> A Motorola S-record file — the bytes, verbatim from the source archive. This is the only file the build reads. |
| <code>SWTBUG.ASM</code> | The period source, kept beside the image so the bytes can be reconciled against the listing. Provenance, not an input.           |
| <code>SWTBUG.LST</code> | The assembler listing, where the archive has one. Provenance.                                                                    |
| <code>DESC</code>       | One hand-written line, shown in <code>SHOW ROMS</code> .                                                                         |
| <code>README.md</code>  | What the ROM is, where it decodes, and where it came from.                                                                       |

The build never assembles `SWTBUG.ASM`. It reads `SWTBUG.S19` and nothing else; the source sits beside the image so a human can check the bytes against it, and so the provenance travels with them.

## How the embed works

`cmake/embed_roms.cmake` looks in each `roms/<NAME>/` for an image — `*.HEX`, `*.BIN`, `*.S19` or `*.SREC` — takes the **first** it finds, and writes it into `roms_generated.cpp` **byte-for-byte**. The extension picks the loader format (`.S19` / `.SREC` → `Format::Srec`, `.HEX` → `Format::Hex`, otherwise raw `Format::Bin`); the C++ side then parses it with the **same loader that backs the `LOAD` command and the monitor's `L`** ([DESIGN.md 10.3.1](#)). The CMake script deliberately does *not* understand any of these formats and must not learn to — the one parser in the binary is the only one there is.

Because it takes the first match, keep **one image per directory**. Ship the `.ASM` / `.LST` for provenance, but exactly one `.S19`.

## Where it lands in memory

A Motorola S-record carries its own load addresses, so the image **self-places** — `builtin:swtbug` decodes to `E000 – E3FF` because that is where its records say to go. Two consequences are worth knowing:

- A ROM region in the machine file gives the *decode window*; the image places itself inside it. `machines/swtpc.toml` mounts `builtin:swtbug` at `E000`, and the records fill `E000 – E3FF`.
- When the same chip is **mirror-decoded** elsewhere — SWTBUG's 1 KB ROM answers the 6800's `FFF8 – FFFF` vector fetch on the real board — the region carries `relocate = true`, which shifts the image so its first record lands at the region's `at` (see the `relocate` key in `docs/manual/ref/boards.md` and the second ROM region in `machines/swtpc.toml`).

## Then wire it in

1. Drop the `.S19` (and the `.ASM/ .LST` provenance) in `roms/<NAME>/` with a one-line `DESC` and a `README.md`. The directory becomes a built-in automatically.
2. Add its CRC32 row to `docs/roms.md` and a case to `tests/test_roms.cpp` — a built-in ROM is a hardware fact, so a mangled embed must fail the **build**, not a user. The test CRC is `crc32(Image::flat())`, i.e. over the `FF`-filled span `lo – hi`, so a part with gaps is `contiguous == false`.
3. Prove it runs. Mount it in a machine and boot: a monitor should reach its prompt and respond to a command over the console. Drive it with `--mcp` (see the [MCP chapter](#)), never a hand-rolled `expect` script.

A worked example, end to end, is `roms/SWTBUG/` — image, source, `DESC`, and the `docs/roms.md` row and `test_roms.cpp` case that guard it.

## If you have source but no image

It has not come up: every built-in so far arrived as a Motorola S-record. If it ever does — an OCR'd 6800 listing with no surviving image — the 6800 path is a period 6800 assembler (TSC's, or FLEX's `ASMB`) emitting an S-record, which then drops in exactly as above. That workflow is **not** mechanised here, and this guide will not pretend it is until someone has actually run it and can write down what bit them. And never commit an image a modern cross-assembler produced without reconciling it, byte-for-byte, against the period listing — the whole point of a built-in ROM is that it is the real bytes.

# Review comments

---

You can leave notes on a document for Claude to act on, without those notes ever touching the tree.

Mark up a copy of a file, ask Claude to address the comments, and the master `.md` in the repository is revised in place. The copy is scratch; the master never carries a marker, so there is nothing to strip out afterward and nothing that can leak into a build.

The mechanism is an HTML comment with a sentinel. HTML comments are invisible in rendered Markdown and in the built PDFs, and they are trivial to `grep` for — the same trick the generated `docs/manual/ref/*.md` banner and the `docs/boards/_TEMPLATE.md` author notes already use.

## The marker

Any HTML comment that begins with `@claude`:

```
Some sentence that reads awkwardly. <!-- @claude: too terse – expand this. -->
```

For a longer note, use the block form:

```
<!-- @claude
This whole section assumes the reader already knows what a hard-sector disk is.
Add a one-paragraph primer before it.
-->
```

When several people are annotating one file, tag who wrote the note:

```
<!-- @claude(patrick): reword this for a first-time user. -->
```

Place the marker next to the text it is about — the sentence it follows, or immediately above the section it refers to. Claude anchors the change to where the marker sits, so position is how you say *what* the comment is about.

## The workflow

1. **Copy** the file you want to review to somewhere outside the repository, keeping its filename. `docs/devguide/theory.md` becomes `~/Desktop/theory.md`, say. The basename is the link back to the master, so do not rename it.
2. **Annotate** the copy: drop `@claude` markers wherever you want a change.
3. **Ask** Claude to act on it — "address the `@claude` comments in `~/Desktop/theory.md`". Claude reads the markers, matches the copy to the repo file of the same name, and revises the **master** to address

each one.

To see every outstanding marker across the tree (they should only ever appear in copies, never here):

```
grep -rn '<!-- @claude' .
```

## What Claude does with them

These are the rules that keep the process safe — worth knowing so you can predict the result:

- **It matches by filename.** The annotated copy shares the master's basename; Claude finds the repo file with that name and edits *that*. If more than one file in the tree has the name (a bare `README.md`, an `ORDER`), Claude asks which one rather than guessing.
- **The copy carries comments, not content.** Your copy may be hours or days out of date. Claude treats the markers — and the text immediately around each one — as the input, and applies them to the *current* master. It does not fold unrelated edits from the copy back into the tree, so a stale copy cannot silently revert someone else's work.
- **It works on a branch**, per the project rule that every change lands on a branch off `master`.
- **There is nothing to clean up.** The master never had a marker, so none is left behind. Claude reports what it changed, comment by comment, and leaves your annotated copy alone — it is yours.

## Annotating in place (the exception)

You *can* drop markers straight into a repo file instead of a copy — handy for a quick note you will resolve in the same sitting. If you do, Claude removes each marker once it is addressed, so none survives the commit.

One place this is not allowed: the **User Manual** (`docs/manual/*.md`). Its raw text — comments included — is grep-checked by `tests/acceptance/docs-manual.cmake` for anything that points outside the shipped package (`src/`, `tests/`, `ctest`, a header name, and so on). A marker that mentions one of those tokens will fail the build even though it never renders. For the manual, review a copy.